



COMSATS University Islamabad, Pakistan

AI-Powered Digital Wardrobe and Outfit Planner (Outfitly)

**A project presented to
COMSATS University Islamabad, Sahiwal Campus**

**In partial fulfillment
of the requirement for the degree of**

Bachelor of Science in Computer Science (2022-2026)

By

Esha Ahmed CIIT/SP22-BCS-036/SWL

Fatima Rizwan CIIT/SP22-BCS-038/SWL

Amna Waseem CIIT/SP22-BCS-128/SWL

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Esha Ahmed

Fatima Rizwan

Amna Waseem

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) “**Outfitly**” was developed by **Esha Ahmed (CIIT/SP22-BCS-036)**, **Fatima Rizwan (CIIT/SP22-BCS-038)** and **Amna Waseem (CIIT/SP22-BCS-128)** under the supervision of “**Dr. Muhammad Shoaib**” and co supervisor “**Mr. Muhammad Jamil**” and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Sciences.

Supervisor

Dr. Muhammad Shoaib

Lecturer

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Prof. Dr. Imran Sarwar Bajwa

Department of Computer Science

The Islamia University of Bahawalpur, Pakistan

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor,

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

Outfitly is an AI-powered Digital Wardrobe and Outfit Planner App designed to help users manage their wardrobe, plan outfits, and make informed fashion choices effortlessly. Traditional wardrobe management is often time-consuming and lacks personalization, while existing digital tools provide limited guidance in outfit coordination and style selection. Outfitly addresses these challenges by enabling users to digitize their clothing, categorize items, and receive AI-driven outfit suggestions based on factors such as colour harmony, style compatibility, and personal preferences. The app leverages machine learning and AI models, including CLIP-based image embeddings for clothing category detection and segmentation-based colour extraction, to automatically analyze wardrobe items and generate intelligent outfit combinations, scoring each by visual compatibility and colour matching.

Users can save their preferred outfits, track clothing usage, and receive styling advice from the AI Fashion Assistant, which provides recommendations through chat or voice input. Outfitly consists of several core modules, the Wardrobe Digitization Module, which allows users to upload, categorize, and manage images of their clothing; the Outfit Suggestions Module, which generates AI-based outfit recommendations; the Writer Module, which enables users to create, add, or remove fashion-related articles, guides, and tips within the app; the Saved Outfits Module, allowing users to save and revisit their favourite outfit combinations.

It is built using Flutter for cross-platform performance and Firebase for secure cloud storage and real-time synchronization, Outfitly ensures a seamless and responsive user experience, while backend AI processing is handled via FastAPI, which communicates with the mobile app to process images, compute embeddings, and return outfit suggestions efficiently. By integrating AI intelligence, content creation, user feedback, and smart wardrobe management, Outfitly simplifies daily outfit selection, promotes sustainable fashion habits, and provides a modern, technology-driven solution for personal fashion management.

Acknowledgment

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “**Dr. Muhammad Shoaib**” and our Co-Supervisor “**Mr. Muhammad Jamil**”. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Esha Ahmed

Fatima Rizwan

Amna Waseem

Abbreviations

SRS	Software Requirements Specification
SDD	Software Design Description
PC	Personal Computer
AI	Artificial Intelligence
ML	Machine Learning
UI	User Interface
UX	User Experience
API	Application Programming Interface
NLP	Natural Language Processing
TTS	Text-to-Speech
SDK	Software Development Kit
APK	Android Package Kit
DBMS	Database Management System
OE	Operating Environment
CO	Constraints
FR	Functional Requirement
GPS	Global Positioning System

Table of Contents

1. Introduction.....	1
1.1. Brief.....	1
1.2. Relevance to Course Modules	2
1.2.1. Artificial Intelligence and Machine Learning.....	2
1.2.2. Software Engineering.....	2
1.2.3. Computer Vision and Image Processing	2
1.2.4. Mobile Application Development.....	2
1.2.5. Database Systems.....	2
1.2.6. Human–Computer Interaction (HCI).....	3
1.3. Project Background.....	3
1.4. Literature Review.....	3
1.5. Analysis from Literature Review	4
1.6. Methodology and Software Lifecycle.....	4
1.6.1. Iterative Development Approach	5
1.6.2. Incremental Delivery and Continuous Feedback.....	5
1.7. Rationale behind Selected Methodology	5
1.7.1. Planning and Requirements Gathering	5
1.7.2. Design	6
1.7.3. Implementation	6
1.7.4. Testing.....	6
1.7.5. Deployment and Maintenance	6
1.8. Rationale behind Selected Approach and Tools.....	6
2. Problem Definition.....	8
2.1. Problem Statement.....	8
2.2. Deliverables Requirements	8
2.2.1. Outfitly Mobile Application	9
2.2.2. Documentation.....	9
2.2.3. Testing Reports	9
2.2.4. Deployment Setup.....	10
2.3. Development Requirements.....	10
2.3.1. Technologies and Tools.....	10

2.3.2.	Server and Hosting Requirements	12
2.3.3.	Performance and Scalability	12
2.3.4.	Security and Privacy	12
2.3.5.	Collaboration and Version Control	12
3.	Requirement Analysis	13
3.1.	Use Case 1.....	13
3.1.1.	Use Case 1 Description.....	13
3.2.	Use Case 2.....	15
3.2.1.	Use Case 2 Description.....	16
3.3.	Use Case 3.....	17
3.3.1.	Use Case 3 Description.....	18
3.4.	Use Case 4.....	19
3.4.1.	Use Case 4 Description.....	20
3.5.	Use Case 5.....	21
3.5.1.	Use Case 5 Description.....	22
3.6.	Use Case 6.....	23
3.6.1.	Use Case 6 Description.....	24
3.7.	Functional Requirements	25
3.7.1.	User Authentication	25
3.7.2.	AI-based Outfit Suggestions	25
3.7.3.	Skin Tone Detection and Colour Matching.....	26
3.7.4.	Virtual Try-On.....	26
3.7.5.	Smart Shopping.....	26
3.7.6.	AI Fashion Assistant	26
3.7.7.	Content Hub and Article Management	27
3.7.8.	Admin Analytics and User Management	27
3.7.9.	Database Module	27
3.8.	Non-Functional Requirements	27
3.8.1.	Usability (USE).....	27
3.8.2.	Performance (PER)	28
3.8.3.	Availability (AVA)	28
3.8.4.	Security (SEC)	28
3.8.5.	Reliability (REL)	28

4. Design and Architecture	29
4.1. System Architecture	29
4.2. Process Flow/Representation	30
4.2.1. User Authentication Flow	30
4.2.2. Wardrobe Management Flow	30
4.2.3. AI-Based Fashion Assistant Flow	31
4.2.4. Virtual Try-on Flow	32
4.2.5. Fashion Content Flow	33
4.2.6. AI Outfit Suggestions Flow	34
4.2.7. Smart Shopping Flow	35
4.3. Design Models	36
4.3.1. Sequence Diagrams.....	37
4.4. Class Diagram.....	43
4.4.1. Class Diagram for Outfitly.....	43
5. Implementation	46
5.1. Algorithm.....	46
5.1.1. User Authentication	46
5.1.2. Wardrobe Upload and Digitization	46
5.1.3. AI Outfit Recommendation.....	47
5.1.4. Shopping Integration Module	47
5.1.5. Weather-Based Outfit Recommendation.....	48
5.1.6. Skin Tone Detection and Suitable Colour Suggestion	48
5.1.7. Fashion Content Module.....	49
5.1.8. Virtual Try-on.....	49
5.1.9. AI Fashion Chatbot.....	50
5.2. External APIs	51
5.3. User Interface.....	52
5.3.1. Home Screen.....	52
5.3.2. User Login Form.....	53
5.3.3. Register Form.....	54
5.3.4. Forgot Password Screen.....	54
5.3.5. User Dashboard.....	55
5.3.6. Wardrobe Management Screen	56

5.3.7.	Add Clothing Item Screen	56
5.3.8.	AI Outfit Recommendation Screen.....	57
5.3.9.	Weather-based Outfit Suggestions Screen	58
5.3.10.	Virtual Try-On Screen.....	58
5.3.11.	Outfit Planner Screen	59
5.3.12.	AI Fashion Assistant Screen	60
5.3.13.	Smart Shopping Screen.....	60
5.3.14.	Content Hub Screen	61
5.3.15.	Skin Tone Detection and Colour Advisor	62
5.3.16.	Fabric Care Advisor	62
5.3.17.	User Profile Screen	63
5.3.18.	User Feedback Screen.....	64
5.3.19.	User Settings Screen	64
5.3.20.	Writer Login and Dashboard Screen.....	65
5.3.21.	Write/Edit Article Screen	66
5.3.22.	SEO Optimization Screen	66
5.3.23.	Writer Settings Screen	67
5.3.24.	Admin Login and Dashboard Screen.....	68
5.3.25.	User Management Screen	68
5.3.26.	Content Management Screen	69
5.3.27.	Manage Shopping Screen	70
5.3.28.	Analytics Screen	70
6.	Testing and Evaluation	72
6.1.	Manual Testing	72
6.1.1.	System Testing.....	72
6.1.2.	Unit Testing	72
6.1.3.	Functional Testing	76
6.1.4.	Integration Testing	77
6.1.5.	Automated Testing	79
7.	Conclusion	81
7.1.	Future Work	81
7.1.1.	Virtual Try-On with 3D Fit and Body Measurement Integration	81
7.1.2.	Advanced AI Personal Stylist	82

7.1.3.	Social Styling and Community Engagement	82
7.1.4.	Region- and Culture-Specific Outfit Suggestions.....	82
7.1.5.	Multi-Language and Regional Support.....	82
8.	References.....	83

List of Tables

Table 1.1: Literature Review	3
Table 2.1: Tools and Technologies	10
Table 3.1: User Authentication and Wardrobe Management	14
Table 3.2: AI-Based Outfit Suggestions	16
Table 3.3: Fashion Content	18
Table 3.4: AI-Based Fashion Assistant.....	20
Table 3.5: Smart Shopping.....	22
Table 3.6: Virtual Try-on.....	24
Table 5.1: External API's	51
Table 6.1: Login as User	73
Table 6.2: User Registration	73
Table 6.3: View Wardrobe Item Details	74
Table 6.4: Generate Outfit Recommendation	74
Table 6.5: Fashion Feed Interaction.....	74
Table 6.6: Schedule Outfit	75
Table 6.7: Add New Wardrobe Item.....	75
Table 6.8: Update User Profile	75
Table 6.9: Create/Edit Article	76
Table 6.10: Logout as User	76
Table 6.11: Functional Testing	76
Table 6.12: Integration Testing.....	78
Table 6.13: Automated Testing.....	79

List of Figures

Figure 1.1: Agile Lifecycle.....	5
Figure 3.1: Use Case of User Authentication and Wardrobe Management.....	13
Figure 3.2: Use Case of AI-Based Outfit Suggestions	16
Figure 3.3: Use Case of Fashion Content	18
Figure 3.4: Use Case of AI-Based Fashion Assistant.....	20
Figure 3.5: Use Case of Smart Shopping.....	22
Figure 3.6: Use Case of Virtual Try-on	23
Figure 4.1: Architecture Diagram	29
Figure 4.2: User Authentication Flow	30
Figure 4.3: Wardrobe Management Flow	31
Figure 4.4: AI-Based Fashion Assistant Flow	32
Figure 4.5: Virtual Try-on Flow	33
Figure 4.6: Fashion Content Flow	34
Figure 4.7: AI Outfit Suggestions Flow.....	35
Figure 4.8: Smart Shopping Flow.....	36
Figure 4.9: Sequence diagram for User Authentication	37
Figure 4.10: Sequence diagram for Wardrobe Management.....	38
Figure 4.11: Sequence diagram for Virtual Try-on	38
Figure 4.12: Sequence diagram for AI-Based Outfit Suggestions.....	39
Figure 4.13: Sequence diagram for Smart Shopping.....	40
Figure 4.14: Sequence diagram for AI-Based Fashion Assistant.....	40
Figure 4.15: Sequence diagram for Fashion Content	41
Figure 4.16: Sequence diagram for Admin.....	42
Figure 4.17: Sequence diagram for Writer	43
Figure 4.18: Class Diagram	45
Figure 5.1: Home Screen	53
Figure 5.2: Login Form.....	53
Figure 5.3: Registration Form.....	54
Figure 5.4: Forgot Password Screen	55
Figure 5.5: User Dashboard	55
Figure 5.6: Wardrobe Management Screen	56

Figure 5.7: Add Clothing Item Screen	57
Figure 5.8: AI Outfit Recommendation Screen	57
Figure 5.9: Weather-based Outfit Suggestions Screen	58
Figure 5.10: Virtual Try-on Screen.....	59
Figure 5.11: Outfit Planner Screen	59
Figure 5.12: AI Fashion Assistant Screen	60
Figure 5.13: Smart Shopping Screen	61
Figure 5.14: Content Hub Screen	61
Figure 5.15: Skin Tone Detection and Colour Advisor	62
Figure 5.16: Fabric Care Advisor	63
Figure 5.17: User Profile Screen.....	63
Figure 5.18: User Feedback Screen	64
Figure 5.19: User Settings Screen.....	65
Figure 5.20: Writer Login and Dashboard.....	65
Figure 5.21: Write/Edit Article Screen	66
Figure 5.22: SEO Optimization Screen	67
Figure 5.23: Writer Settings Screen.....	67
Figure 5.24: Admin Dashboard	68
Figure 5.25: User Management Screen.....	69
Figure 5.26: Content Management Screen	69
Figure 5.27: Manage Shopping Screen.....	70
Figure 5.28: Analytics Screen.....	71

Chapter 1

INTRODUCTION

1. Introduction

Outfitly is an AI-powered digital wardrobe and outfit planning application designed to transform the way users manage their clothing and make fashion decisions. Many people struggle with organizing their wardrobe, choosing daily outfits, and maintaining sustainable fashion habits, while existing tools often provide limited support [1]. Outfitly addresses these challenges by integrating digital wardrobe organization, AI-based outfit recommendations, virtual try-on, fabric care guidance, shopping suggestions, and a smart AI Fashion Assistant into a single, intelligent platform. By offering personalized outfit suggestions based on user preferences, weather, occasions, and clothing usage patterns, Outfitly simplifies decision-making and helps users look their best effortlessly.

It is built using Flutter and Firebase, Outfitly ensures smooth cross-platform performance, real-time synchronization, and secure cloud storage. Its AI Fashion Assistant learns individual style preferences over time, providing tailored advice and innovative outfit combinations. By tracking clothing usage and offering insights for sustainable choices, the application encourages smarter, more eco-friendly fashion decisions. Overall, Outfitly transforms traditional wardrobe management into a dynamic, efficient, and intelligent experience, making fashion organization easier, more personalized, and environmentally conscious for users of all ages.

1.1. Brief

This project presents Outfitly, an AI-powered mobile application developed to simplify wardrobe management and enhance personal styling through intelligent automation. The system integrates secure user authentication, digital wardrobe organization, AI-based outfit recommendations, a fashion content hub, AI fashion assistance, smart shopping integration, and a virtual try-on feature. Using tools such as Flutter, FastAPI, CLIP, Whisper, which are discussed later in Chapter 6, Outfitly provides personalized styling suggestions, fabric care guidance, and curated fashion content for everyday users, fashion enthusiasts, and sustainability-focused individuals which are discussed later in Chapter 3. The methodology includes requirement analysis, system design, AI model integration, backend and frontend development, followed by testing and evaluation. The project outcome is a fully functional, modular fashion assistant capable of improving user styling decisions, optimizing wardrobe usage, and delivering an interactive and engaging user experience. Additionally, the system adapts to user preferences for a truly personalized fashion experience.

1.2. Relevance to Course Modules

The project directly integrates and applies concepts learned across multiple courses in the Bachelor of Computer Science (BCS) program. It demonstrates how theoretical knowledge can be transformed into a practical, real-world fashion management application. Additionally, the project reinforces software engineering principles like system design, modular development, and API integration, providing a comprehensive learning experience that bridges academic theory with industry-relevant practice.

1.2.1. Artificial Intelligence and Machine Learning

Core AI and ML algorithms are utilized for generating personalized outfit recommendations, recognizing clothing patterns, and predicting fashion trends. Concepts from these courses guided the design of intelligent modules that adapt to user preferences, and style choices.

1.2.2. Software Engineering

Principles of requirement analysis, system design, modular development, and Agile-based SDLC are applied throughout the project. These practices ensured structured development, maintainability, and scalability of the mobile application. Continuous iteration and feedback cycles further improved feature refinement and overall system quality.

1.2.3. Computer Vision and Image Processing

Modules such as skin tone detection and dominant colour extraction, leverage computer vision techniques. Knowledge from image processing courses was applied to process user-uploaded images, detect clothing items, analyze colours, and improve the relevance of outfit suggestions.

1.2.4. Mobile Application Development

The project leverages Flutter and Dart to build a responsive, interactive, and cross-platform mobile application. Skills from mobile app development courses were essential in creating a smooth user experience with real-time interactions.

1.2.5. Database Systems

Firebase is used for efficient data storage, retrieval, and synchronization across devices. Database design concepts from the course helped structure user profiles, wardrobe data, outfit recommendations, and shopping information in a reliable and optimized way. Additionally, Firestore's real-time updates enable seamless interaction between the mobile app and backend, ensuring users always see the most current data.

1.2.6. Human–Computer Interaction (HCI)

HCI principles shaped the app’s intuitive and user-friendly interface, guiding layout, navigation, and interaction flow. These usability techniques ensured accessibility and an engaging experience for all users, translating theoretical concepts into a practical, user-centred fashion management application.

1.3. Project Background

This project originated from the common challenges people face in organizing their wardrobes and selecting outfits [15]. Many individuals struggle to coordinate their clothing, often repeating outfits or forgetting items they own. While existing wardrobe apps allow manual cataloguing, they lack intelligence, personalization, and integration with factors like weather, trends, or occasion-based styling.

The Outfitly address these gaps by using AI and machine learning to deliver a smarter, more personalized fashion experience. Users can digitize their wardrobe through image uploads, after which the system categorizes items and generates tailored outfit suggestions for various events and conditions. This approach promotes efficient wardrobe management, informed styling decisions, and sustainable fashion habits.

1.4. Literature Review

Current research trends emphasize the use of AI-driven personalization and data-driven sustainability in fashion technology. However, there remains a lack of comprehensive systems that combine wardrobe digitization, outfit recommendations [6], smart shopping assistance, and sustainability tracking into one integrated platform, a gap this project seeks to fill. By addressing these gaps, the project contributes to advancing intelligent fashion management solutions. This project draws inspiration from several existing wardrobe and outfit management applications, some of which are summarized in Table 1.1.

Table 1.1: Literature Review

App	Weaknesses	Proposed Solution
Clozzie	Focuses on wardrobe organization; lacks AI personalization, sustainability insights, and real-time data.	Add AI-driven outfit recommendations, sustainability tracking, and real-time trend integration.

Acloset	Offers outfit suggestions but lacks multi-store coordination, deep personalization, and sustainability features.	Enable multi-store shopping, AI personalization, and sustainability tracking.
Shkaf	Manual wardrobe management; no AI or real-time trend integration; limited sustainability features.	Implement AI recommendations, real-time data integration, and sustainability insights.

1.5. Analysis from Literature Review

The comparative analysis reveals that most existing systems tend to focus exclusively on either wardrobe organization or outfit recommendations, often addressing only a single aspect of fashion management. Few systems effectively combine both functionalities, and critical features such as sustainability, fabric-care guidance, and wardrobe efficiency tracking are frequently overlooked [9]. The proposed system sets itself apart by offering a comprehensive and intelligent solution that integrates multiple key components:

- AI-driven outfit recommendations tailored to the user’s personal style, weather conditions, specific occasions.
- Smart shopping assistance that consolidates options across multiple stores, helping users make informed purchasing decisions without the hassle of browsing numerous platforms.
- Sustainable fashion insights and wardrobe efficiency tracking, enabling users to adopt environmentally conscious choices while maximizing the usability and lifespan of their clothing items.

By seamlessly integrating these features, the proposed system goes beyond the limited scope of existing solutions, providing a truly intelligent assistant that supports users in every aspect of their fashion journey, from daily outfit planning to sustainable wardrobe optimization [3].

1.6. Methodology and Software Lifecycle

For the development of the Outfitly project, the Agile methodology has been chosen. Agile allows for iterative development, rapid prototyping [10], and continuous feedback from users, which is essential for a fashion-focused, interactive application like Outfitly. This methodology

ensures flexibility, adaptability, and active collaboration between the development team and potential users, enabling a user-centric approach and faster delivery of functional modules.

1.6.1. Iterative Development Approach

Outfitly is developed in iterative sprints, with each sprint focusing on modules such as Wardrobe Digitization, Outfit Suggestions, AI Fashion Articles [4], Fashion Assistant, and Virtual Try-On. Following Agile principles, the team collaborates closely to define user stories, prioritize features, and plan sprint backlogs. This approach ensures flexibility, quick adaptation to change, and continuous improvement based on user feedback.

1.6.2. Incremental Delivery and Continuous Feedback

Outfitly uses incremental development, where each module is completed and fully functional before moving to the next, with thorough testing and user evaluation at every stage. This incremental approach reduces risks, detects issues early, and ensures new features enhance the overall experience while aligning with user needs and long-term goals.

1.7. Rationale behind Selected Methodology

The Figure 1.1 illustrates the Agile methodology selected for Outfitly that promotes iterative and incremental development, allowing features to evolve progressively through continuous user feedback. Agile enables the team to respond quickly to changing requirements, implement feedback effectively, reduce the risk of major errors, and detect issues early through short iterative cycles.

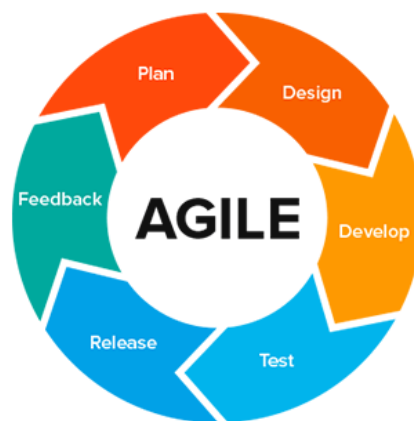


Figure 1.1: Agile Lifecycle

1.7.1. Planning and Requirements Gathering

During this phase, the requirements for each feature were gathered and prioritized based on user needs. This includes understanding functionality for modules such as Wardrobe

Digitization, Outfit Suggestions [6], Smart Shopping Integration, and Virtual Try-on. User stories and acceptance criteria were defined to guide development.

1.7.2. Design

A detailed design was created for each module, defining the data structures, user interface, workflows, and interactions between components. Agile encourages lightweight, flexible design that can evolve as user feedback is collected. Object-oriented principles were applied where applicable to ensure reusability, maintainability, and organized code.

1.7.3. Implementation

Modules were implemented incrementally during each sprint, starting with a basic functional version and refined based on feedback and testing. Continuous integration ensured each new module integrated smoothly into the application.

1.7.4. Testing

Testing was performed throughout development, including unit testing, integration testing, and user acceptance testing (UAT). Continuous testing facilitated early identification of issues and maintained a reliable, bug-free user experience.

1.7.5. Deployment and Maintenance

At the end of each sprint, completed features are deployed in a functional version of the application. Maintenance is ongoing and involves fixing bugs, refining features, and incorporating new user requirements or enhancements. Agile ensures that the application evolves continuously while remaining stable and user-friendly.

1.8. Rationale behind Selected Approach and Tools

The project uses Object-Oriented Programming (OOP) principles for modularity, reusability, and scalability. Each major feature (recommendations, fashion chatbot, shopping) is designed as an independent module interacting through defined interfaces.

Selected Tools and Technologies:

- Flutter (v3.35) and Dart (v3.4): For building the front-end mobile interface with responsive layouts and smooth user interactions.
- Firebase: Used for user authentication, data storage, and analytics.

- Python (v3.12) with Hugging Face Transformers: For implementing AI-based recommendation and NLP functionalities.
- Figma (v125.4.9): For UI/UX mock-up design and user flow visualization.
- MS Project (v2021): For project scheduling and Gantt chart creation.
- MS Word (v2021): Used to create and format project documentation such as SRS, design documents, and final reports.

These technologies collectively support cross-platform compatibility, real-time synchronization, AI integration, and high performance, enabling the project to deliver a modern, intelligent, and user-friendly wardrobe management solution.

Chapter 2

PROBLEM DEFINATION

2. Problem Definition

This addresses the challenges of managing and organizing clothing, planning daily outfits, and promoting sustainable fashion choices. Using AI and ML, the system provides personalized outfit recommendations, skin tone detection, and intelligent style guidance to simplify clothing management and enhance user convenience.

2.1. Problem Statement

In today's digital age, individuals often struggle to manage their wardrobes effectively, leading to repeated outfits, clutter, and inefficient use of clothing. Existing solutions rely on manual organization and fail to provide personalized outfit suggestions based on factors like weather, event type, colour compatibility, and user preferences [6]. They also rarely address sustainability, such as tracking underused garments or recommending eco-friendly shopping options. The absence of an integrated, intelligent system results in the following major issues:

- Users waste time deciding what to wear due to lack of AI-driven assistance.
- Wardrobe items remain underutilized because they are not efficiently categorized or recommended.
- Consumers make redundant or impulsive purchases, leading to increased wardrobe waste.
- No centralized system combines fashion assistant, outfit recommendations, and sustainability tracking.

Therefore, there is a need for an AI-based digital wardrobe system that provides intelligent outfit recommendations, wardrobe organization, and sustainable fashion insights, all within a user-friendly mobile application.

2.2. Deliverables Requirements

The deliverables of the AI-Powered Digital Wardrobe and Outfit Planner application are structured to ensure the successful completion of all functional, technical, and user-oriented objectives throughout the project lifecycle. These deliverables include working software components, documentation artifacts, user interfaces, and deployment-ready modules. Each deliverable is designed to meet the system goals of intelligent styling assistance, wardrobe digitization, personalization, and seamless user experience.

2.2.1. Outfitly Mobile Application

Outfitly is an AI-powered mobile application that simplifies wardrobe management and enhances personal styling by digitizing clothing items and providing personalized outfit suggestions based on factors like colour, occasion, weather, and season. Its key modules include AI-driven outfit recommendations, virtual try-on for previewing clothing on the user, a fashion chatbot offering real-time style advice, skin tone detection for suitable colour suggestions, and smart shopping recommendations tailored to the user's style and wardrobe needs [4]. Additional features include fabric care guidance, a colour palette advisor, and fashion articles. The application is designed to be highly interactive, engaging users with intuitive features and seamless navigation. Overall, Outfitly aims to save time, optimize outfit choices, and improve the user's styling experience.

2.2.2. Documentation

The project includes user guides for navigating the app and using its features, technical documentation covering system architecture and AI modules, and an administrator guide for managing backend services and updates.

- **User Documentation:** A complete guide explaining how users can navigate through the application, upload wardrobe items, generate outfit suggestions, use the fabric care advisor, and track clothing usage, and interacting with the fashion chatbot.
- **Technical Documentation:** It covers the system architecture, database schema (Firestore), AI modules (Hugging Face models, CLIP), APIs, model training processes, and overall system workflow.
- **Administrator Guide:** A guide for administrators or developers explaining how to manage backend services, update AI models, maintain databases, fix bugs, and deploy new updates.

2.2.3. Testing Reports

Comprehensive testing will be conducted to ensure the quality and reliability of the Outfitly application. This includes unit tests for individual modules, integration tests for system components, and User Acceptance Testing (UAT) to ensure Outfitly meets quality standards and user expectations. The results of these tests will help identify and resolve potential issues, ensuring a stable, efficient, and user-friendly application.

2.2.4. Deployment Setup

The project delivers a deployable Android app, with potential iOS deployment in the future. Deployment includes setup, API integration, security configurations, backups, and guidelines for updates and scalability [6]. The app will be prepared for release through platforms such as the Google Play Store or Firebase App Distribution for testing purposes.

2.3. Development Requirements

The development of the Outfitly mobile application is guided by modern development practices, cloud-based architecture, and AI integration to ensure high performance, scalability, and personalized user experiences. This section outlines the technical, server-side, security, and performance requirements essential for the successful implementation of the system.

2.3.1. Technologies and Tools

The Table 2.1 contains environment configuration settings and technology stacks used for the Outfitly Mobile Application, including backend frameworks, databases, APIs, AI integrations, and other supporting tools that ensure system performance, scalability, and reliability.

Table 2.1: Tools and Technologies

Tools and Technologies	Tools	Version	Rationale
Frontend Framework	Flutter	3.x	Flutter is used for building the cross-platform mobile application with a smooth UI and native-like performance for Android and iOS.
Backend Framework	Firebase	Latest	Firebase provides backend services such as Firestore database, authentication, storage, and real-time synchronization.
Database	Firebase Firestore	Latest	Firestore is used for storing user data, wardrobe items, articles, comments, and feedbacks in real time.

Authentication Service	Firebase Authentication	Latest	Handles secure user login and registration for different roles (User, Writer, Admin). Supports email/password and Google sign-in.
AI Services	Hugging Face TinyLlama	Latest	A lightweight AI model integrated for chatbot responses and outfit recommendations efficiently.
Skin Tone Detection	Python CNN	4.x	Custom Convolutional Neural Network trained on a skin tone dataset; predicts user skin tone to provide suitable colour suggestions.
Smart Shopping API	RapidAPI/ Custom Web Scraper	Latest	Fetches related fashion items and product links from partner e-commerce websites for Smart Shopping recommendations.
Version Control	Git/GitHub	Latest	Used for managing project versions, collaboration, and code integrity.
Design Tools	Figma / Adobe XD	Latest	Used for UI/UX design of mobile screens, user flow diagrams, and prototype visualization.
Testing Framework	Flutter Test / Firebase Emulator Suite	Latest	Used for unit, widget, and integration testing to ensure the reliability and stability of all application modules.
Analytics Tool	Firebase Analytics	Latest	Tracks user behaviour, wardrobe usage patterns, and engagement metrics to improve personalized recommendations.
Analytics (Server / Custom)	Vercel Analytics API	Latest	Server-side event collection, custom analytics endpoints and lightweight ML analytics hosted on Vercel.

Deployment Platform	Google Play Console	Latest	Used for distributing and managing the Outfitly Android application.
Token Management	JWT (JSON Web Tokens)	Latest	Ensures secure communication between the client and backend through access and refresh tokens.

2.3.2. Server and Hosting Requirements

The application is hosted and managed using Firebase Hosting and Firebase Firestore backend, ensuring efficient real-time data access and secure handling of user-generated content [11]. All communication is secured using HTTPS and SSL encryption. No external database or cloud storage is used, as Firestore manages all structured and unstructured data.

2.3.3. Performance and Scalability

Real-time data handling is achieved through Firestore's automatic synchronization capabilities. Performance is optimized using indexed queries, caching mechanisms, and image resolution control. The application is designed to scale automatically as the number of users and wardrobe entries increases. API responses from OpenWeather, Hugging Face, and Vercel are processed efficiently to ensure low latency and fast AI recommendation generation.

2.3.4. Security and Privacy

Security is enforced using Firebase Authentication with secure login protocols. Firestore security rules ensure user data isolation and restrict unauthorized access. All sensitive information is encrypted, and API keys are securely managed through environment configuration. The application follows privacy best practices, ensuring user wardrobe and preference data remain confidential.

2.3.5. Collaboration and Version Control

The development process is managed using Git and GitHub for source code control, branching, version tracking, and collaborative development.

Chapter 3

REQUIREMENT ANALYSIS

3. Requirement Analysis

The requirement analysis outlines the expected behaviour, user interactions, and operational flow of the AI-powered wardrobe and outfit planner. It defines functional requirements like wardrobe digitization, AI outfit recommendations, and weather-based suggestions, along with non-functional aspects such as performance, scalability, security, and usability. Use case models and diagrams guide system design, help identify constraints, and ensure features align with user needs [5]. This structured approach also supports planning, resource allocation, and thorough testing to ensure the system functions as intended.

3.1. Use Case 1

In Figure 3.1, the use case diagram shows User Authentication and Wardrobe Management in Outfitly, with two actors: User and Admin. Users can authenticate, upload, categorize, edit, delete, and favourite clothing items, while Admin manages user accounts. The diagram illustrates actor interactions and role-based operations within the wardrobe system.

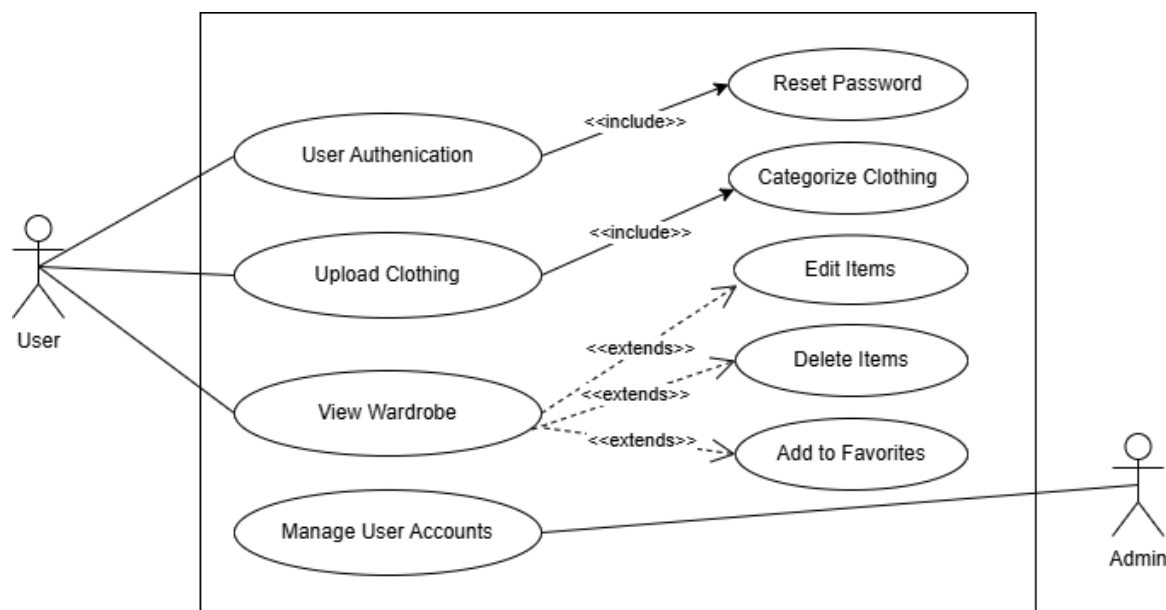


Figure 3.1: Use Case of User Authentication and Wardrobe Management

3.1.1. Use Case 1 Description

The Table 3.1 outlines the authentication, wardrobe management, and content interaction processes in Outfitly. It describes how users register or log in, upload, categorize, edit, delete, and mark clothing items as favourites, while administrators manage user accounts. The table provides a clear overview of system functionalities and the interactions between different actors.

Table 3.1: User Authentication and Wardrobe Management

Use Case ID	UC-1
Use Case Name	User Authentication and Wardrobe Management
Actors	Primary Actor: User Secondary Actor: Admin
Description	This use case describes how users authenticate, upload clothing, categorize items, view wardrobe items, edit or delete items, and add favourites. The administrator manages user accounts.
Trigger	The user initiates authentication, uploads clothing, or interacts with wardrobe items. The administrator performs account management actions.
Preconditions	PRE-1. The user must have an internet connection. PRE-2. User must be registered or able to register. PRE-3. User must be registered or able to register.
Postconditions	POST-1. The user is logged in or successfully registered. POST-2. Admin updates to user accounts are saved.
Normal Flow	1. User Authentication i. User selects Login or Register. ii. System verifies details and grants access. 2. Reset Password i. User selects Forgot Password. ii. System prompts for PIN or Security Question set during registration. iii. User enters PIN or answers the security question. iv. System verifies the input allows user to set a new password. 3. Upload Clothing (include Categorize Clothing) i. User selects Add Clothing Item. ii. User captures or selects an image. iii. System uploads the image. iv. System prompts for categorization and system saves it. 4. View Wardrobe i. User opens View Wardrobe. ii. System displays clothing items details. 5. Edit Items (extends View Wardrobe) i. The user clicks the "Edit Item" button on their clothing item.

	ii. System updates the data. 6. Delete Items (extends View Wardrobe) i. The user clicks the "Delete Item" button on their clothing item. ii. System deletes the clothing item. 7. Add to Favorites (extends View Wardrobe) i. The user clicks the heart icon on the clothing item details screen. ii. System saves the preference. 8. Manage User Accounts (Admin) i. Admin opens Manage User Accounts. ii. System shows the user list. iii. Admin modifies, block, or delete users.
Alternative Flows	1. Authentication - Wrong Credentials i. User enters invalid credentials. ii. System displays an error. iii. User retries. 2. Upload Clothing - No Category Selected i. User attempts to save item without category. ii. System prompts for category selection.
Exceptions	1.0. E1 If credentials are incorrect, an error message is displayed. 2.0. E2 Unsupported file type/size, an error message is displayed.
Business Rules	Users must be authenticated to access wardrobe features. Only admins can manage users.
Assumptions	1. Users will enter correct details. 2. Internet connection is stable for uploads. 3. Users understand basic clothing categories.

3.2. Use Case 2

Figure 3.2 shows how users interact with Outfitly's AI-Based Outfit Suggestions. Users select wardrobe items like shirts, trousers, pants, or shoes to generate personalized outfit recommendations and view multiple suggested combinations. Administrators monitor AI outputs for quality, ensuring accurate and relevant recommendations, while the system adapts suggestions over time based on user preferences and trends.

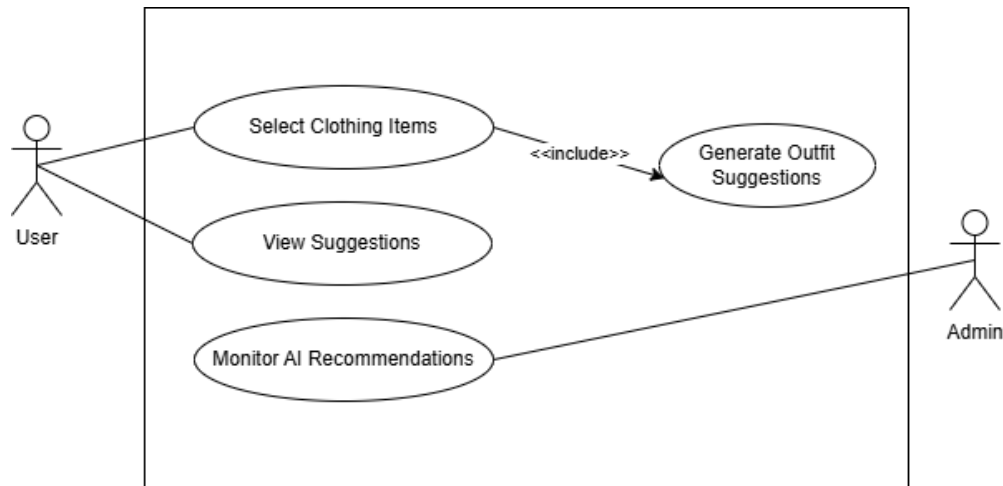


Figure 3.2: Use Case of AI-Based Outfit Suggestions

3.2.1. Use Case 2 Description

The Table 3.2 outlines the AI-Based Outfit Suggestions processes in Outfitly. It describes how users select clothing items from their wardrobe (mandatory shirts and pants, optional shoes) to generate personalized outfit recommendations. Users can view suggestions, while administrators monitor AI outputs to ensure accuracy and quality.

Table 3.2: AI-Based Outfit Suggestions

Use Case ID	UC-2
Use Case Name	AI-Based Outfit Suggestions
Actors	Primary Actor: User Secondary Actor: Admin
Description	Users upload one or more clothing items (e.g., shirts, pants) and optionally shoes. The AI generates matching outfit suggestions with reasoning. Admin monitors AI outputs for accuracy.
Trigger	User wants outfit recommendations.
Preconditions	PRE-1. User must be authenticated. PRE-2. At least one clothing item (shirt/pant) is uploaded. PRE-3. AI recommendation engine is available.
Postconditions	POST-1. AI generates suggested outfits. POST-2. Suggestions can be viewed by user.
Normal Flow	1. Select Clothing Items i. User opens AI Outfit Suggestions module. ii. User selects a shirt (mandatory).

	iii. User selects one or more pants (mandatory). iv. User optionally selects shoes. 2. Generate Outfit Suggestions (extends Select Clothing Item) i. System sends selected items to AI engine. ii. AI generates outfit combinations with reasoning. iii. System provides top suggestions for user. 3. View Suggestions i. User browses suggested outfits. 4. Monitor AI Recommendations i. Admin reviews AI suggestions. ii. Admin adjusts recommendations if needed.
Alternative Flows	1. Select Clothing Items - Insufficient Items Selected i. User selects fewer than required items (no shirt or pant). ii. System prompts user to select mandatory items. 2. Monitor Recommendations - AI Engine Error i. System cannot generate suggestions due to processing or server error. ii. System notifies user to retry later.
Exceptions	1.0. E1 Uploaded images are blurry or unsupported, system requests new images. 2.0. E2 AI cannot generate a match, system suggests alternative items or outfits.
Business Rules	Admin can monitor and adjust AI outputs to maintain accuracy. Shoes are optional but recommended for complete outfit combinations.
Assumptions	1. Internet connection is stable for AI processing. 2. Users understand how to select clothing items correctly. 3. Admin follows organizational policies during AI monitoring.

3.3. Use Case 3

Figure 3.3 illustrates how users, writers, and administrators interact with Outfitly's Fashion Articles module. Users can browse, read, like, comment, and save articles; writers upload and manage content; and administrators approve, publish, or moderate articles. It also demonstrates

how the system supports collaborative engagement between users and content creators, ensuring an interactive and dynamic platform for fashion insights.

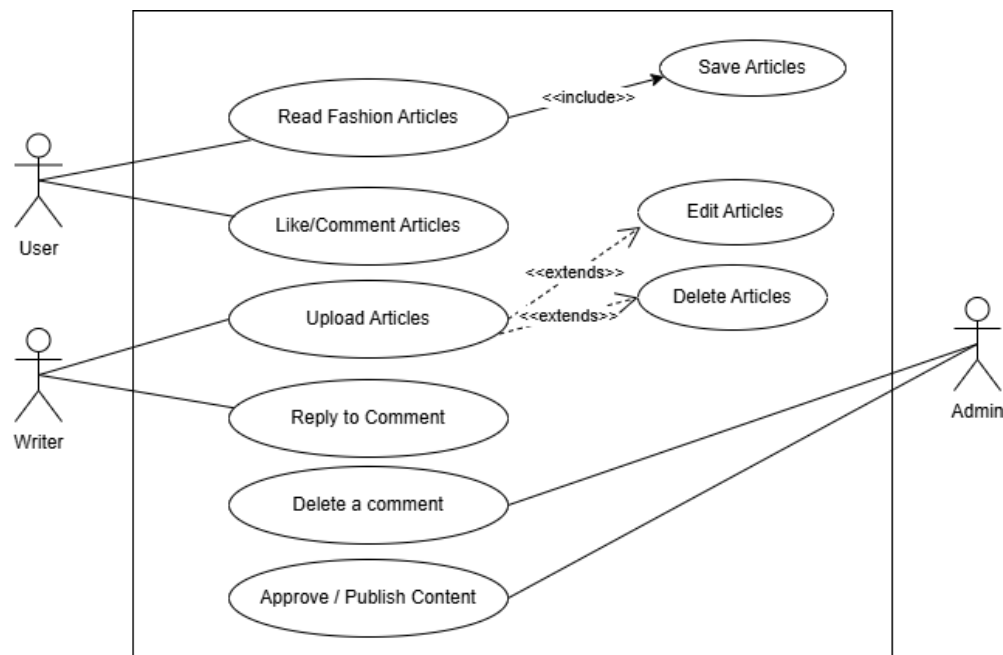


Figure 3.3: Use Case of Fashion Content

3.3.1. Use Case 3 Description

The Table 3.3 outlines the Fashion Articles processes in Outfitly. It describes how users browse, read, like, comment on, and save articles; writers upload and manage content; and administrators approve, publish, or moderate articles. The table provides a clear overview of system functionalities and the interactions between different actors within the content hub.

Table 3.3: Fashion Content

Use Case ID	UC-3
Use Case Name	Fashion Content
Actors	Primary Actor: User, Writer Secondary Actor: Admin
Description	Users can read, like, comment, and save articles. Writers can upload new articles and manage comments, while administrators approve or publish content.
Trigger	The actor initiates actions such as reading an article, interacting with content, uploading an article, or moderating posts.
Preconditions	PRE-1. Writers and administrators must be authenticated. PRE-2. Articles must exist for reading or interaction.

Postconditions	POST-1. New content is approved and published. POST-2. Comments are added, replied to, or deleted.
Normal Flow	1. Read Fashion Articles (include Save Article) - User selects an article to read. - System loads and displays the article. - User chooses to save the article; system stores it. 2. Like/Comment Articles (User) - User likes an article or writes a comment. - System records the interaction. 3. Upload Articles (Writer) - Writer selects Upload Article. - Writer enters article text, images, and metadata. - System submits article for approval. 4. Monitor AI Recommendations (Admin) - Admin reviews AI suggestions. - Admin adjusts recommendations if needed.
Alternative Flows	1. Upload Articles - Incomplete Details - Writer submits incomplete content. - System requests missing fields. 2. Edit Article – Unauthorized Edit - Writer attempts editing an article not owned by them. - System denies editing.
Exceptions	1.0. E1 System cannot load the article, an error displayed. 2.0. E2 Upload fail due to network or server issue.
Business Rules	Only writers can upload and edit their own articles. Users can like/comment only on published articles.
Assumptions	1. Users and writers have stable internet access. 2. Writers submit meaningful and valid content.

3.4. Use Case 4

Figure 3.4 shows how users and administrators interact with Outfitly's AI-Based Fashion Assistant. Users can ask queries via text or voice, receive responses in text or audio, and

optionally view generated outfit images. Administrators monitor AI interactions to ensure quality, highlighting the system’s role in providing interactive, personalized fashion guidance.

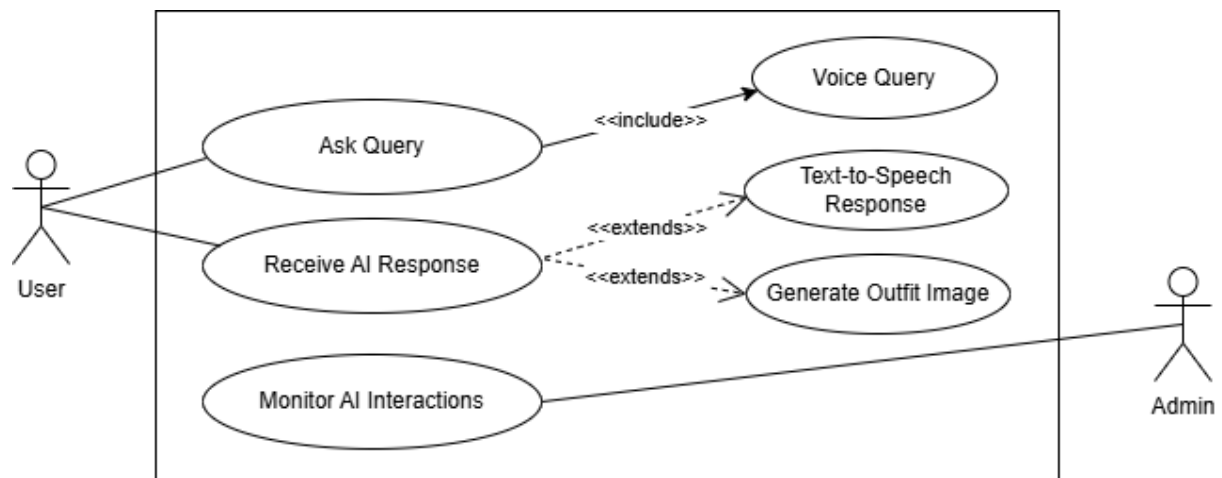


Figure 3.4: Use Case of AI-Based Fashion Assistant

3.4.1. Use Case 4 Description

The Table 3.4 outlines the AI Fashion Assistant processes in Outfitly. It describes how users interact with the AI by typing or speaking queries, receiving text or voice responses, and optionally generating outfit images. Administrators monitor AI interactions to ensure accuracy, and proper functioning.

Table 3.4: AI-Based Fashion Assistant

Use Case ID	UC-4
Use Case Name	AI-Based Fashion Assistant
Actors	Primary Actor: User Secondary Actor: Admin
Description	Users interact with the AI system by asking queries, receiving responses, using voice input, and generating outfit recommendations.
Trigger	The user initiates a query or interacts with the AI, the administrator accesses monitoring tools.
Preconditions	PRE-1. User must have access to the system. PRE-2. Network connection must be active.
Postconditions	POST-1. The user receives an AI-generated response, either text or voice. POST-2. Optional outfit images are generated.
Normal Flow	1. Ask Query (include Voice Query) i. User enters a query in the chat interface.

	ii. System receives and processes the query. iii. User can speak the query and system converts voice to text and proceeds. 2. Receive AI Response (User) i. System generates a text-based AI response. ii. System displays the response to the user. 3. Text-to-Speech Response (extends Receive AI Response) i. User enables audio output by clicking speaker icon. ii. System converts AI text response to speech. 4. Generate Outfit Image (extends Receive AI Response) i. Ask to generate image for fashion-related queries. ii. AI generates an outfit image suggestion. 5. Monitor AI Interactions (Admin) i. Admin accesses the AI interaction logs. ii. Admin reviews or reports inappropriate or inaccurate AI behaviour.
Alternative Flows	1. Ask Query - Voice Query Not Recognized i. System cannot interpret the spoken query. ii. System prompts the user to retry or type the query. 2. Generate Outfit Image - Outfit Image Unavailable i. AI cannot generate image. ii. System returns a standard text response instead.
Exceptions	1.0. E1 System stops processing, displays an error, and waits for reconnection. 2.0. E2 AI fails to process the query, system informs user of temporary unavailability.
Business Rules	Only authenticated users can submit queries to the AI. Voice queries and AI image generation must follow resource limits.
Assumptions	1. Internet connection is stable for AI processing. 2. Devices used support text, audio, or image output.

3.5. Use Case 5

Figure 3.5 shows how users interact with Outfitly's Smart Shopping module by selecting categories, viewing brands, and visiting brand websites. Administrators manage listings,

ensuring accuracy and smooth shopping experiences. The diagram highlights user interactions and the seamless navigation between categories and brand options.

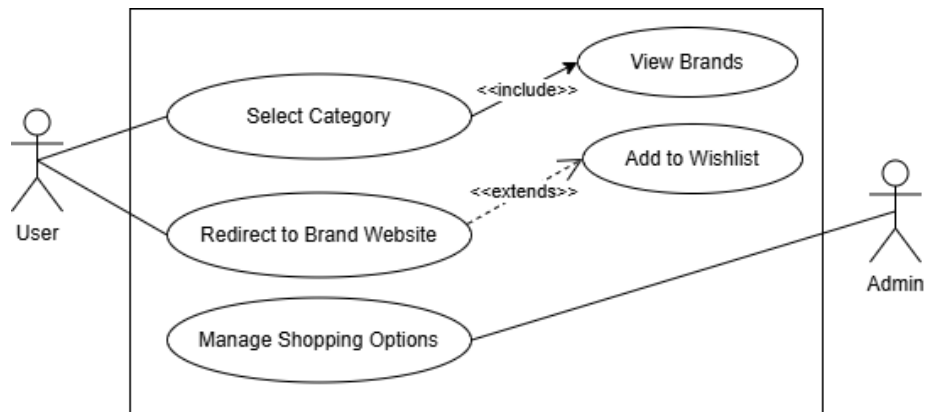


Figure 3.5: Use Case of Smart Shopping

3.5.1. Use Case 5 Description

The Table 3.5 outlines the Smart Shopping processes in Outfitly. It describes how users select a clothing category, view brands, and are redirected to brand websites for purchase. Administrators manage and update brand listings for a seamless shopping experience.

Table 3.5: Smart Shopping

Use Case ID	UC-5
Use Case Name	Smart Shopping
Actors	Primary Actor: User Secondary Actor: Admin
Description	Users select shopping categories, view available brands and redirect to brand websites.
Trigger	User selects a category, wants to browse brands, or accesses shopping options.
Preconditions	PRE-1. User must have access to the system. PRE-2. Network connection must be active.
Postconditions	POST-1. User successfully views brands and redirects to sites. POST-2. Admin changes to shopping options are saved.
Normal Flow	1. Select Category (include View Brands) - User chooses a shopping category. - System displays available category options. - System shows brands available in the selected category.

	2. Redirect to Brand Website (User) i. User clicks a selected brand. ii. System redirects user to the official brand website. 3. Manage Shopping Options (Admin) i. Admin accesses the AI interaction logs. ii. Admin reviews or reports inappropriate or inaccurate AI behaviour.
Alternative Flows	1. Select Category - Invalid Category Selection i. User selects a category with no brands. ii. System displays “No brands available”. 2. Redirect to Brand Website - Failed Brand Redirect i. Brand website fails to open. ii. System displays error and suggests retry.
Exceptions	1.0. E1 System cannot load brands or redirect; user is notified. 2.0. E2 Non-admin tries accessing admin shopping options; system denies.
Business Rules	User must select a valid category before viewing brands.
Assumptions	1. Users understand basic shopping categories. 2. Users expect accurate redirection to official brand websites.

3.6. Use Case 6

Figure 3.6 shows how users interact with Outfitly’s Virtual Try-On module by uploading clothing items, using the camera to overlay outfits, adjusting positions, and taking snapshots. It allows users to visualize outfits and make confident style choices.

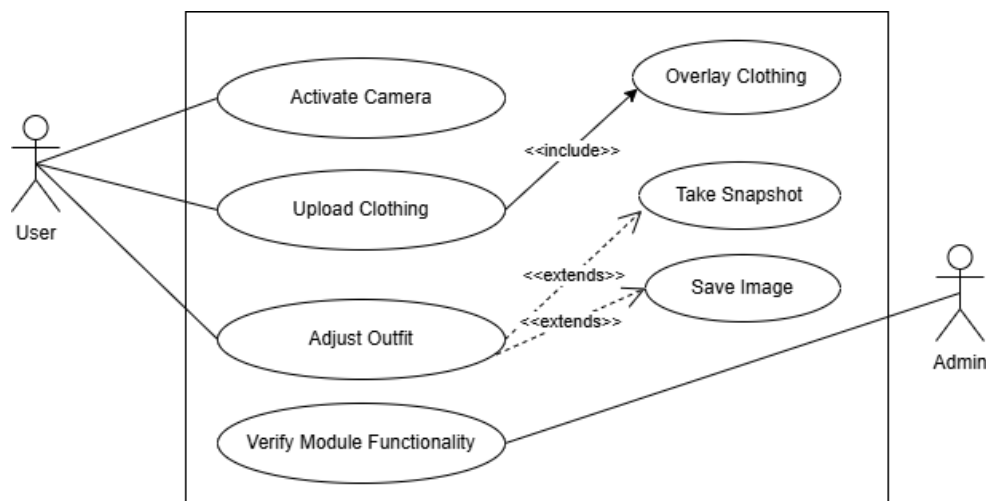


Figure 3.6: Use Case of Virtual Try-on

3.6.1. Use Case 6 Description

The Table 3.6 outlines the Virtual Try-On processes in Outfitly. It describes how users upload clothing items, activate the camera, overlay outfits, adjust positions, and take snapshots to visualize combinations. The module helps users experiment with styles before wearing or purchasing.

Table 3.6: Virtual Try-on

Use Case ID	UC-6
Use Case Name	Virtual Try-on
Actors	Primary Actor: User Secondary Actor: Admin
Description	User activate the camera, upload clothing, overlay outfits on their image, adjust the outfit, take snapshots, and save the results.
Trigger	User activates the camera, uploads clothing, or attempts to try on an outfit.
Preconditions	PRE-1. Camera permission must be granted. PRE-2. User must have clothing uploaded or accessible.
Postconditions	POST-1. Outfit overlay is generated. POST-2. User may capture or save final try-on image.
Normal Flow	1. Activate Camera (User) i. User opens the Virtual Try-on module. ii. System activates the device camera and displays the user preview. 2. Upload Clothing (include Overlay Clothing) i. User selects "Upload Clothing". ii. System overlays the uploaded clothing on the user's body. iii. The preview is displayed in real-time. 3. Adjust Outfit (User) i. User adjusts clothing position, size, and angle. ii. System updates overlay accordingly. 4. Take Snapshot (Extends Adjust Outfit) i. User captures the current try-on view. ii. System saves the image temporarily for preview. 5. Save Image (Extends Adjust Outfit) i. User chooses to permanently save the snapshot.

	ii. System stores the image in user's gallery storage. 6. Verify Module Functionality (Admin) i. System checks camera feed, and output generation. ii. Admin logs or reports any issues.
Alternative Flows	1. Activate Camera - Camera Permission Denied i. User denies camera access. ii. System displays permission request again. 2. Upload Clothing - Clothing Upload Failure i. Invalid format or corrupted image. ii. System prompts user to re-upload a valid clothing image.
Exceptions	1.0. E1 System takes too long to generate overlay and operation fails. 2.0. E2 Saving snapshot fails due to insufficient memory.
Business Rules	User must upload valid image formats (JPG, PNG). Try-on overlay must not proceed without a clothing image.
Assumptions	1. User understands how to position themselves for camera-based try-on. 2. Uploaded clothing images are clear and well-lit.

3.7. Functional Requirements

This section defines the functional requirements of the Outfitly application, which describe the core system functionalities from the perspective of user interaction, system behaviour, and role-based responsibilities.

3.7.1. User Authentication

The User Authentication Module enables secure registration and login for Outfitly. Users can register with email and password, select a role (User or Writer), or use Google Sign-In. A PIN or security question is stored in Firestore for account recovery. Separate login screens exist for Users, Writers, and Admins, providing role-based access to features [9]. Password recovery is supported via PIN, security question, or Google account. Sessions are managed securely to prevent unauthorized access, ensuring safe and role-specific access across the app.

3.7.2. AI-based Outfit Suggestions

The AI-Based Outfit Suggestions module allows users to upload images of shirts, pants, and shoes, which are analyzed using AI models via a Hugging Face API. The system leverages the CLIP model for category detection and embeddings, along with segmentation and KMeans-

based dominant colour extraction to evaluate clothing items. Personalized outfit combinations are generated based on colour harmony, style compatibility, occasion, and user preferences. Each suggestion is scored for visual compatibility, with the top-ranked outfits returned to the user.

3.7.3. Skin Tone Detection and Colour Matching

The Skin Tone Detection module allows users to upload a photo of their face or exposed skin area. The system uses a MobileNet-V2 model, trained on a skin tone dataset from Kaggle and deployed via Hugging Face, to classify the user's skin tone into categories such as Dark, Light, Dark-Mid, and Light-Mid. Based on the detected skin tone, the module suggests clothing colours that complement the user's complexion [10]. These personalised recommendations refine outfit choices by prioritizing colours suited to the user's skin tone.

3.7.4. Virtual Try-On

The Virtual Try-On module enables users to overlay selected clothing items on a live camera feed or uploaded photo. Users can adjust the position, scale, and orientation of clothing items to visualize how outfits look on their body. This feature increases engagement by allowing users to preview their outfit combinations realistically before wearing or purchasing items. Only pre-approved clothing images with transparent backgrounds are used to ensure accurate overlay.

3.7.5. Smart Shopping

The Smart Shopping module enables users to explore and purchase wardrobe items that complement their existing outfits. Upon opening the module, users are presented with clothing categories such as Eastern Wear, Western Wear, Shoes, and Accessories. Selecting a category displays relevant brands that offer products matching the user's style, colour preferences, and outfit needs. Users can browse products, view detailed product information, and add items to a wishlist. When a user selects a product, they are redirected to the official brand's website for purchase, providing a seamless, informed, and convenient shopping experience.

3.7.6. AI Fashion Assistant

The AI Fashion Assistant offers real-time style guidance, outfit coordination tips, and personalized fashion advice through chat or voice input. It leverages a custom-trained TinyLLaMA model, built using a self-curated fashion dataset and deployed via the Hugging Face API, to understand user queries accurately [8]. The assistant processes natural language

and speech inputs using NLP and Whisper-based speech recognition, providing context-aware responses. It offers advice on outfit combinations, clothing care, and shopping recommendations, and evolving fashion trends to enhance the overall user experience.

3.7.7. Content Hub and Article Management

The Content Hub allows content writers to upload, edit, and manage fashion-related articles, including outfit tips, seasonal trends, and styling guides. Admins moderate the content before publishing to ensure quality and relevance. The module supports SEO optimization by enabling writers to add relevant keywords, tags, and meta descriptions, enhancing visibility and discoverability of content within the app. Users can browse content by category and save articles for later reading.

3.7.8. Admin Analytics and User Management

Admins can monitor app usage metrics and AI model performance. This module provides reports while ensuring data privacy. Admins also manage user roles and permissions, block or remove accounts, and ensure proper system security. The module supports role-based dashboards, providing efficient tools to maintain and monitor overall system functionality.

3.7.9. Database Module

The Database Module manages the secure storage and retrieval of all application data, including user profiles, wardrobe items, outfit suggestions, AI-generated recommendations, shopping information, and fashion content articles. It is implemented using Firebase Firestore, enabling real-time synchronization across devices and seamless CRUD operations. Sensitive data, such as passwords and security information, is encrypted to ensure privacy and protection.

3.8. Non-Functional Requirements

This section defines the quality attributes of the AI-Powered Digital Wardrobe and Outfit Planner App, ensuring performance, security, maintainability, and usability.

3.8.1. Usability (USE)

The Outfitly application will employ a responsive design to ensure accessibility across a range of devices, including smartphones, tablets, and varying screen sizes. A consistent and visually appealing colour scheme will be maintained throughout the app to enhance user experience. Additionally, Font Awesome icons will be incorporated where appropriate to improve accessibility and provide clear visual cues, helping users navigate the application intuitively.

3.8.2. Performance (PER)

The system is designed to handle up to 500 wardrobe items per user without any noticeable performance degradation. Asynchronous operations will be employed to fetch multiple responses simultaneously from the server and external services, ensuring efficiency. The user interface will remain responsive during intensive operations, such as filtering, sorting, or applying AI-generated suggestions, preventing freezes or delays in interaction.

3.8.3. Availability (AVA)

Outfitly will be available 24/7, leveraging cloud-based hosting services to ensure minimal downtime. Automatic failover mechanisms will be implemented to manage server crashes or disruptions without interrupting service for end users. The application is designed to operate efficiently across multiple operating systems and modern web browsers, providing a reliable and accessible experience for all users.

3.8.4. Security (SEC)

User passwords will be securely stored using cryptographic hashing techniques, ensuring that they remain unreadable without the corresponding salt. JSON Web Tokens (JWT) will be used for secure session management, preventing unauthorized access [8]. All user inputs will be sanitized to protect against common vulnerabilities, including SQL injection and cross-site scripting (XSS) attacks, ensuring a secure environment for application data and interactions.

3.8.5. Reliability (REL)

The AI-based outfit recommendation system will be trained to achieve an accuracy of at least 85%, providing dependable guidance for users. Daily automated backups will be performed to prevent data loss and corruption, while system recovery protocols are designed to restore functionality within 10 minutes in the event of an unexpected failure. These measures collectively ensure that Outfitly remains reliable, consistent, and trustworthy for all users.

Chapter 4

DESIGN AND ARCHITECTURE

4. Design and Architecture

The overall design and architectural framework of the AI-Powered Digital Wardrobe and Outfit Planner App (Outfitly) define how system components interact, how data is represented and processed, and how the design models support the application's functionality. The architecture ensures scalability, modularity, and seamless integration between the user interface, database, and AI modules.

4.1. System Architecture

The Figure 4.1 describes system follows a modular, multi-tier architecture that integrates AI processing, cloud storage, and mobile client interaction to deliver an intelligent fashion experience. Users interact with the system through a mobile interface built to manage wardrobe items, receive recommendations, and access fashion content [14]. The core of the system consists of six interconnected modules: User Authentication & Wardrobe Management, AI-Based Outfit Suggestions, Content Hub, AI Fashion Assistant, Smart Shopping Integration, and Virtual Try-On. These modules rely on advanced AI models for vision analysis, natural language processing, speech recognition, and recommendation generation [9]. All user data, wardrobe images and articles, are securely stored and processed in cloud infrastructure.

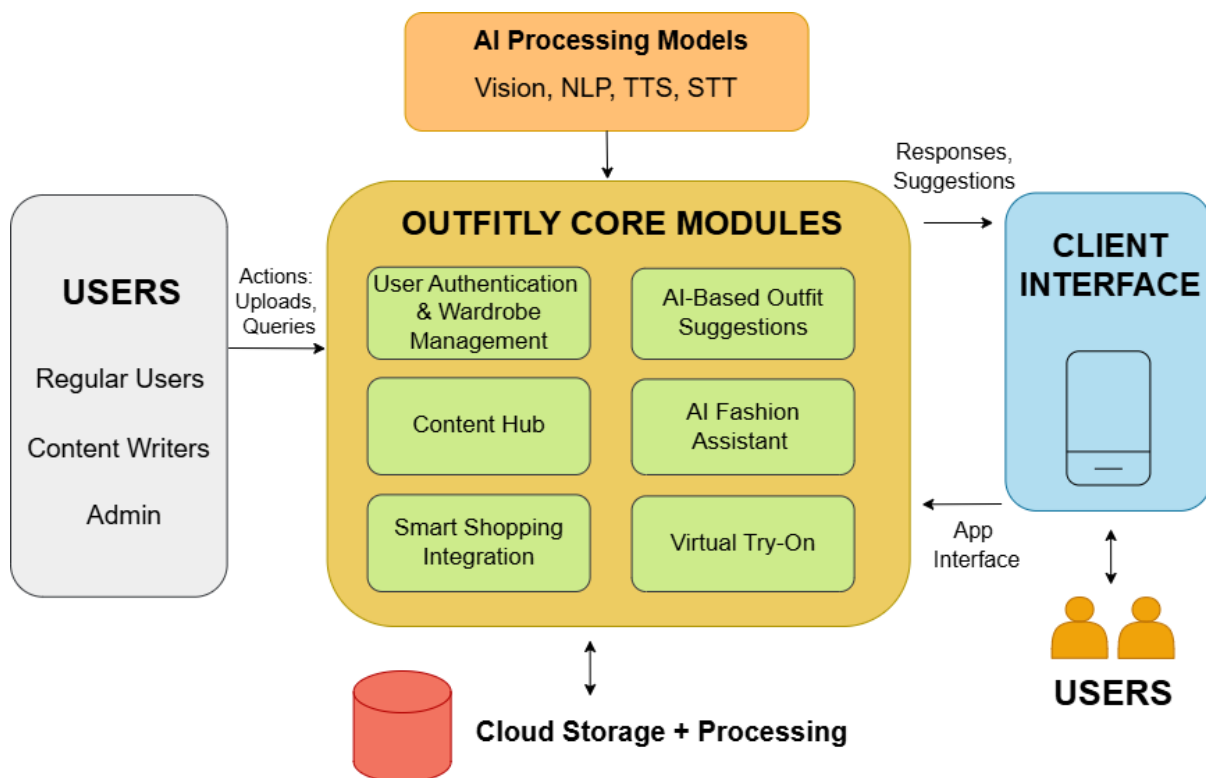


Figure 4.1: Architecture Diagram

4.2. Process Flow/Representation

The Outfitly process flows begin with user registration and authentication, enabling access to features like wardrobe digitization, AI-based outfit suggestions, virtual try-on, and smart shopping [13]. Users upload clothing images, which the system processes through categorization. They can plan outfits, receive personalized recommendations, interact with the AI Fashion Assistant through text or voice, and explore brands through the shopping module.

4.2.1. User Authentication Flow

The Figure 4.2 illustrates the user authentication and wardrobe management workflow of the AI-powered wardrobe application. The process begins with user registration or login, ensuring secure access to the system. Once authenticated, users can digitize their clothing items by uploading images and entering relevant details. The system organizes these items into categories to build a personalized digital wardrobe.

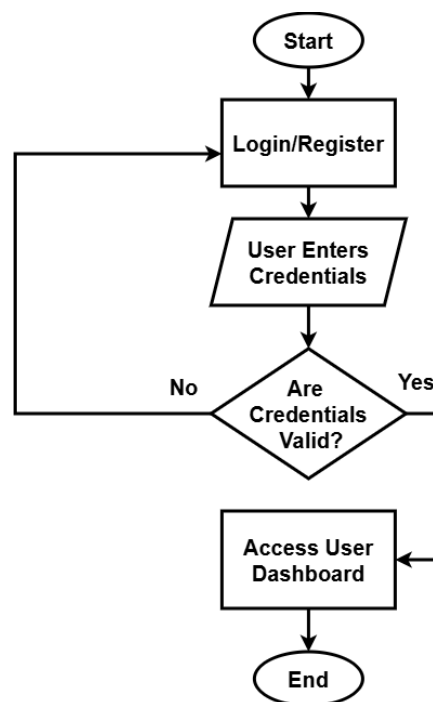


Figure 4.2: User Authentication Flow

4.2.2. Wardrobe Management Flow

The Figure 4.3 describes the wardrobe management process flow describes how users interact with the AI wardrobe app: digitize their clothing, and organize items. The app then helps browse, select, and plan outfits, providing personalized recommendations based on preferences, weather, occasions, and fashion trends.

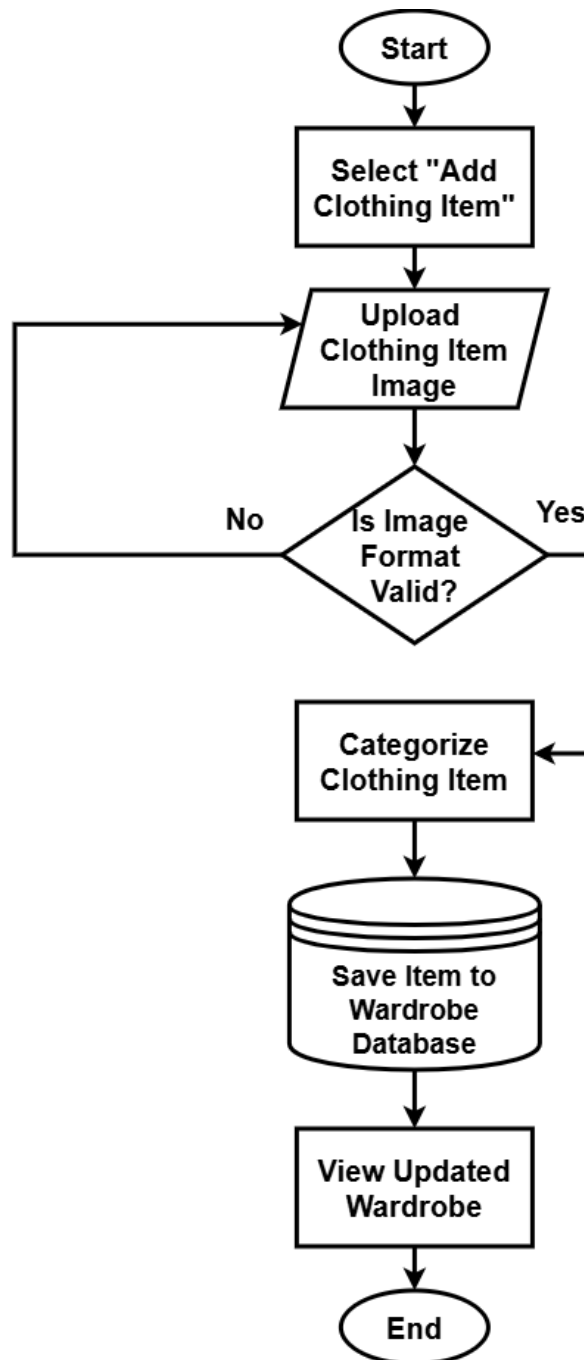


Figure 4.3: Wardrobe Management Flow

4.2.3. AI-Based Fashion Assistant Flow

The Figure 4.4 describes the AI-based Fashion Assistant flow guides users through an interactive, intelligent styling experience. Users can ask fashion-related questions through text or voice, view outfit suggestions, and receive AI-generated styling tips. It can also generate outfit images, suggest matching items, and offer fabric care tips, creating a personalized virtual stylist experience.

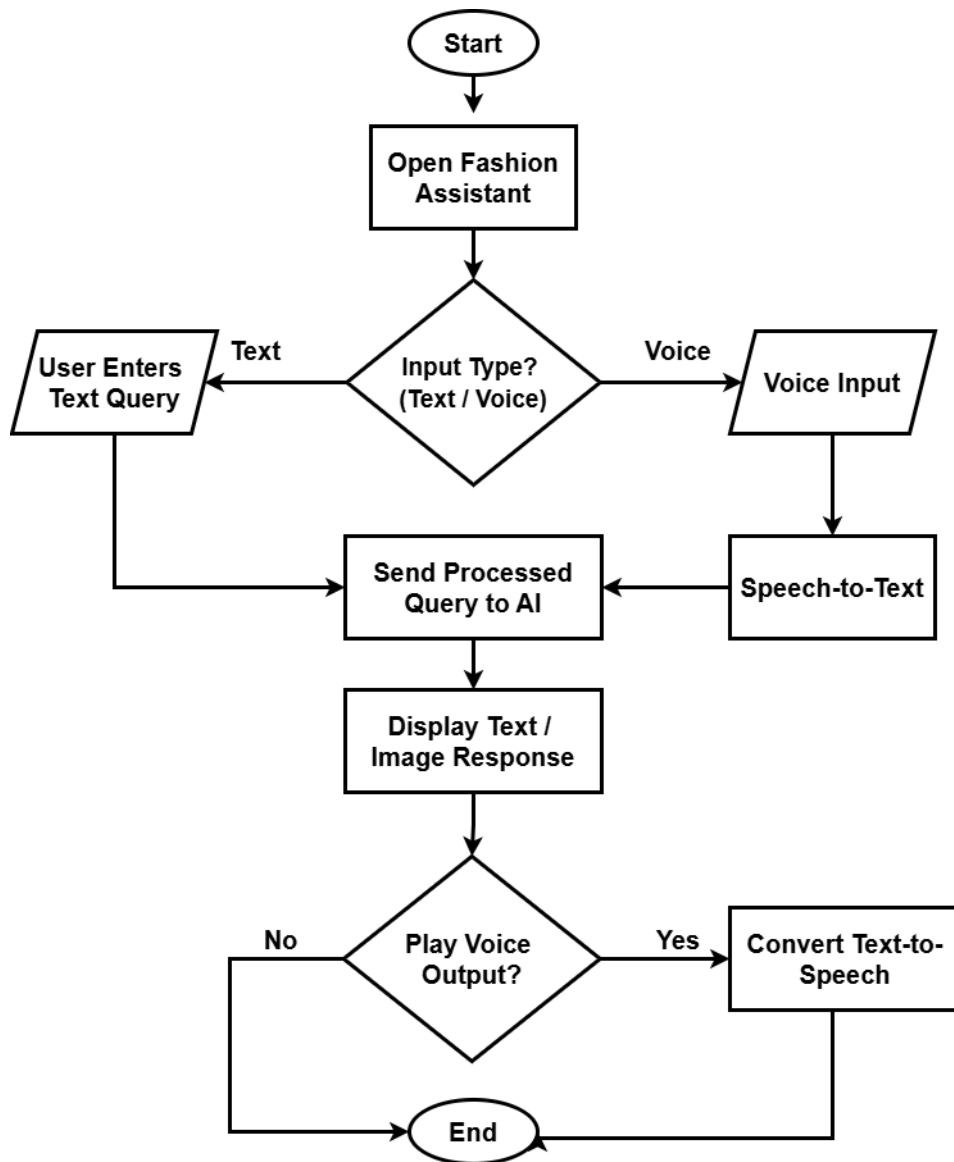


Figure 4.4: AI-Based Fashion Assistant Flow

4.2.4. Virtual Try-on Flow

The Figure 4.5 illustrates the Virtual Try-On workflow, which demonstrates how users interact with the feature to preview outfits on a digital model. Users upload clothing item images from their wardrobe, and the system realistically overlays the selected outfit onto a virtual representation. The generated try-on result allows users to view, compare, and evaluate different outfit combinations before finalizing their choice. Additionally, users can save preferred looks for future reference or discard unsuitable options. This flow enhances user experience by reducing uncertainty in styling decisions, increasing confidence in outfit selection, and supporting more informed fashion choices through visual simulation.

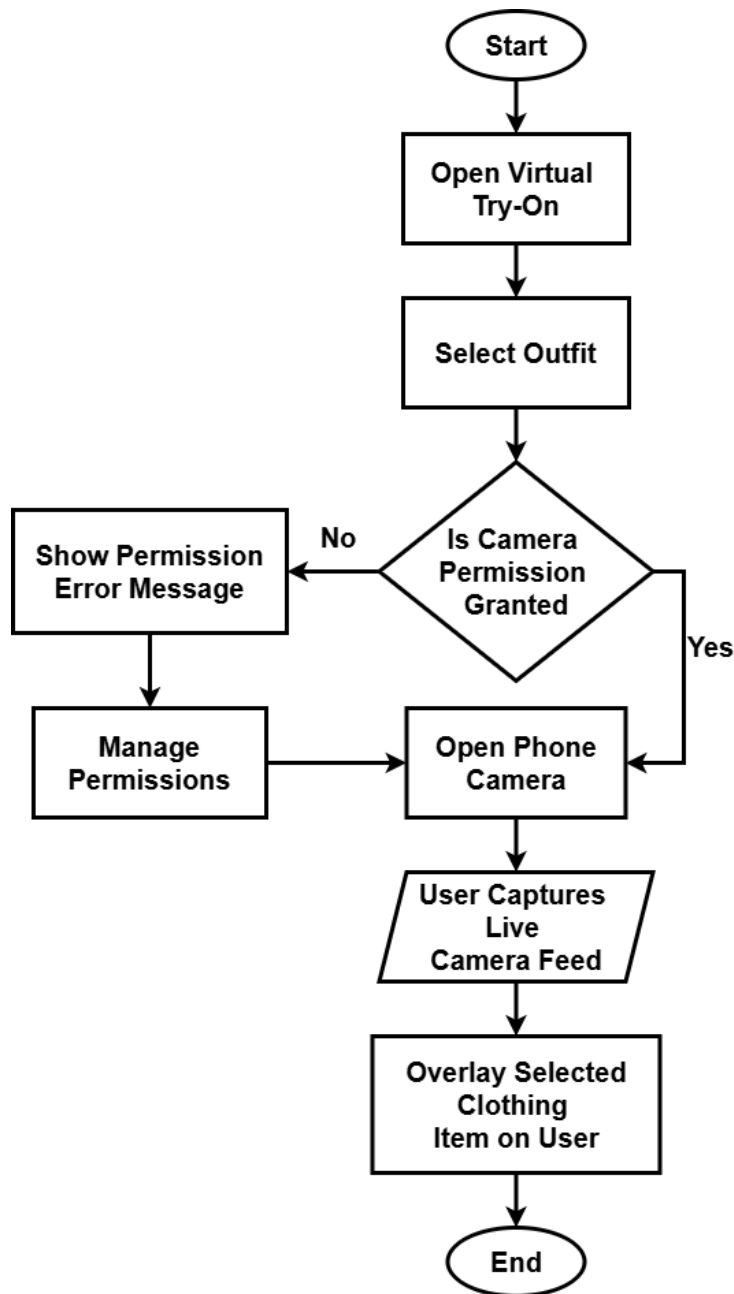


Figure 4.5: Virtual Try-on Flow

4.2.5. Fashion Content Flow

The Figure 4.6 describes the Fashion Articles module allows users to read AI-curated and writer-submitted fashion content. The flow starts when the user opens the Articles section, where the system fetches the latest articles from the database. Articles are categorized by trends, styling tips, seasonal guides, and brand reviews. When a user selects an article, the system loads the full content along with images, related articles, and recommendations [6]. Users can like, save, comment, or share the article. The module continuously updates based on new submissions from writers and trending fashion topics.

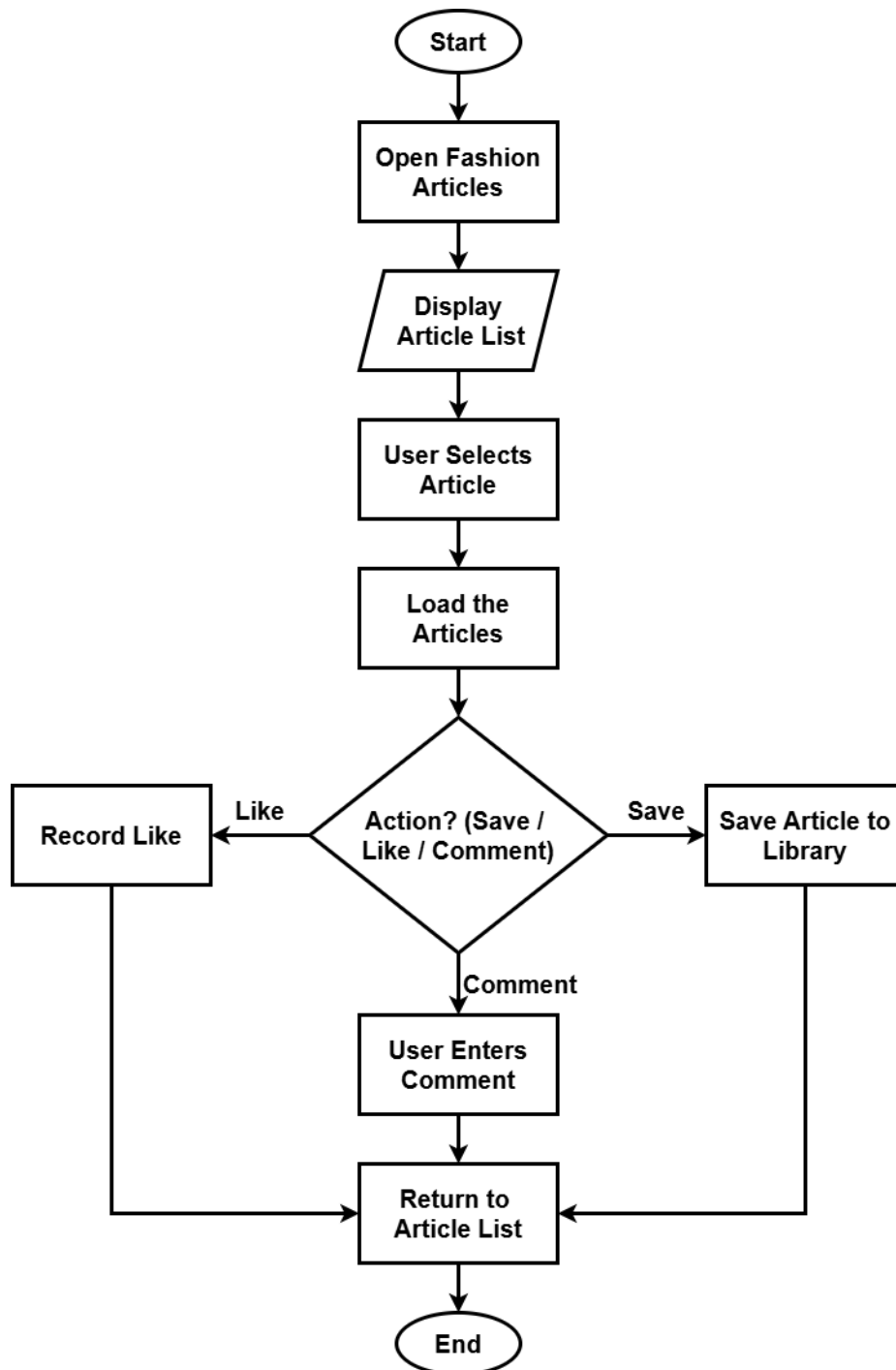


Figure 4.6: Fashion Content Flow

4.2.6. AI Outfit Suggestions Flow

The Figure 4.7 describes the AI Outfit Suggestions flow starts when the user selects items or chooses an occasion, weather, or style. The system analyses the user's wardrobe and trends to generate personalized outfit combinations [15]. Users can view, save, or request alternative looks, ensuring fast and tailored fashion recommendations.

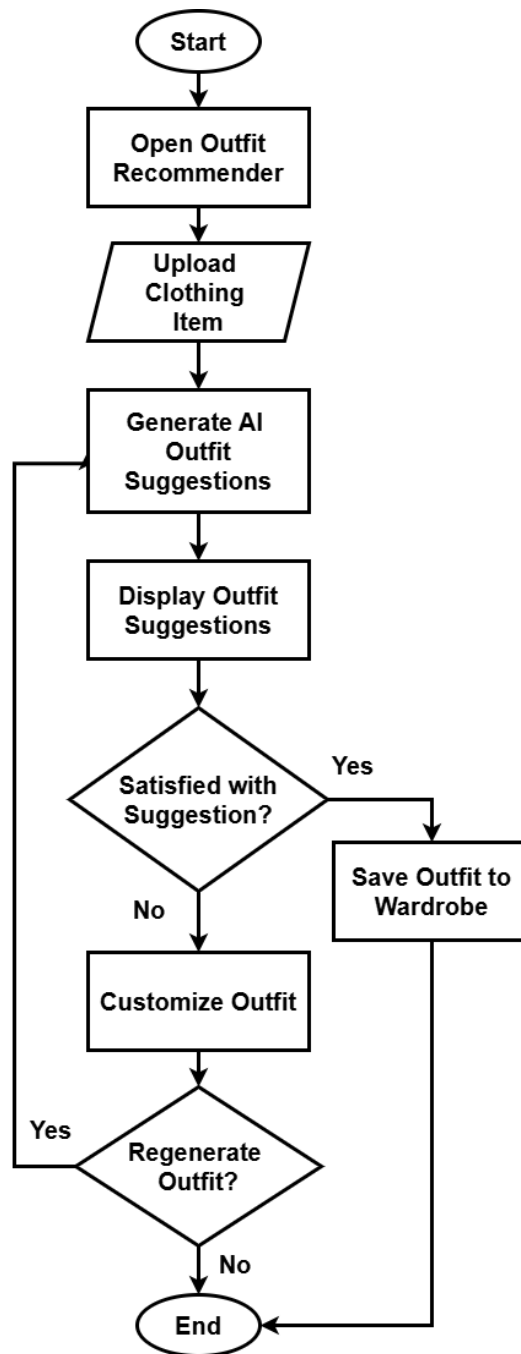


Figure 4.7: AI Outfit Suggestions Flow

4.2.7. Smart Shopping Flow

The Figure 4.8 describes the Smart Shopping feature helps users discover clothing items that match their wardrobe gaps, personal style, and upcoming needs. It also highlights budget-friendly options, ongoing sales, and sustainable choices. This module ensures a seamless, personalized, and efficient shopping experience.

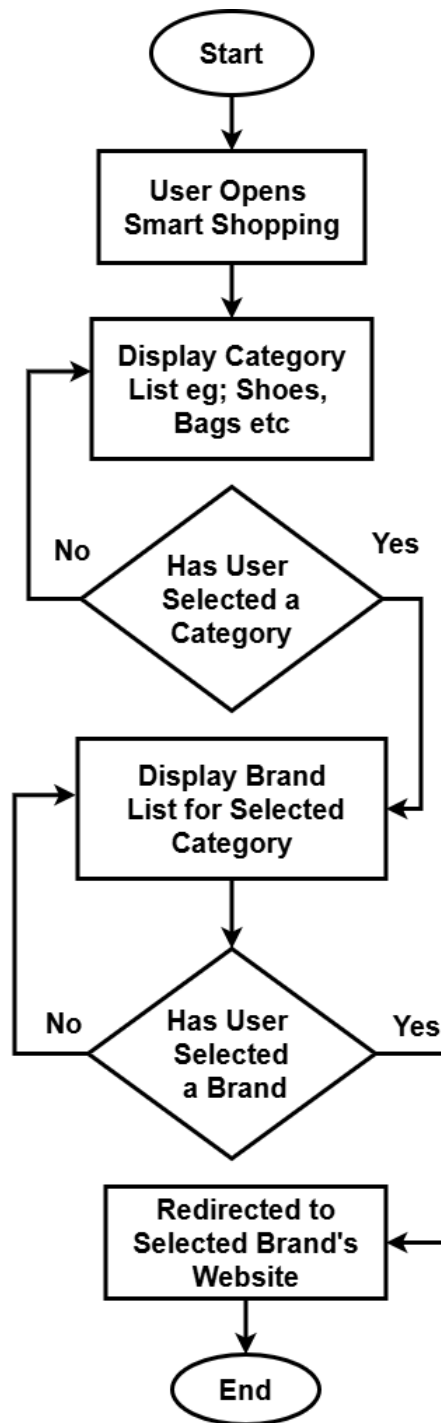


Figure 4.8: Smart Shopping Flow

4.3. Design Models

It is essential to include design models that offer a detailed and visual representation of the system's architecture, user interactions, and module communication [17]. These models support clear understanding among stakeholders, guide developers through system functionality, and reduce ambiguity in implementation.

4.3.1. Sequence Diagrams

Sequence Diagrams are interaction diagrams that detail how operations are carried out. They capture the interaction between objects in the context of a collaboration [13]. Sequence Diagrams are time focus, and they show the order of the interaction visually by using the vertical axis of the diagram to represent time what messages are sent.

4.3.1.1. Sequence diagram for User Authentication

The Figure 4.9 illustrates how a user log into the Outfitly app. The user enters their login credentials, which are sent to the authentication service. The service verifies the credentials and responds with a success or failure message. If authentication is successful, the user is granted access to the app's features [14].

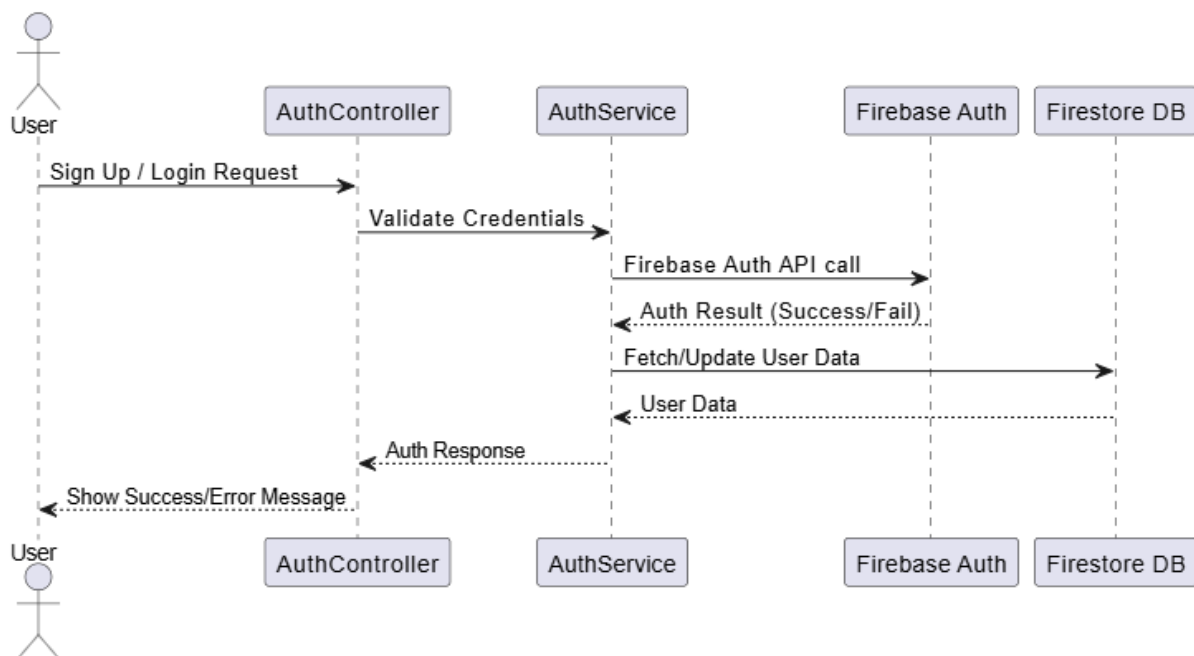


Figure 4.9: Sequence diagram for User Authentication

4.3.1.2. Sequence diagram for Wardrobe Management

The Figure 4.10 illustrates how a user manages their wardrobe within the app. After logging in, the user can upload clothing images. The system processes these images, stores them in the wardrobe database, and updates the wardrobe view. After successfully saving the new item, the system retrieves the updated list of wardrobe items from the database. This ensures that the user interface always reflects the most recent wardrobe state. Finally, the updated wardrobe data is returned to the user, allowing them to view, organize, and effectively manage their clothing collection in real time.

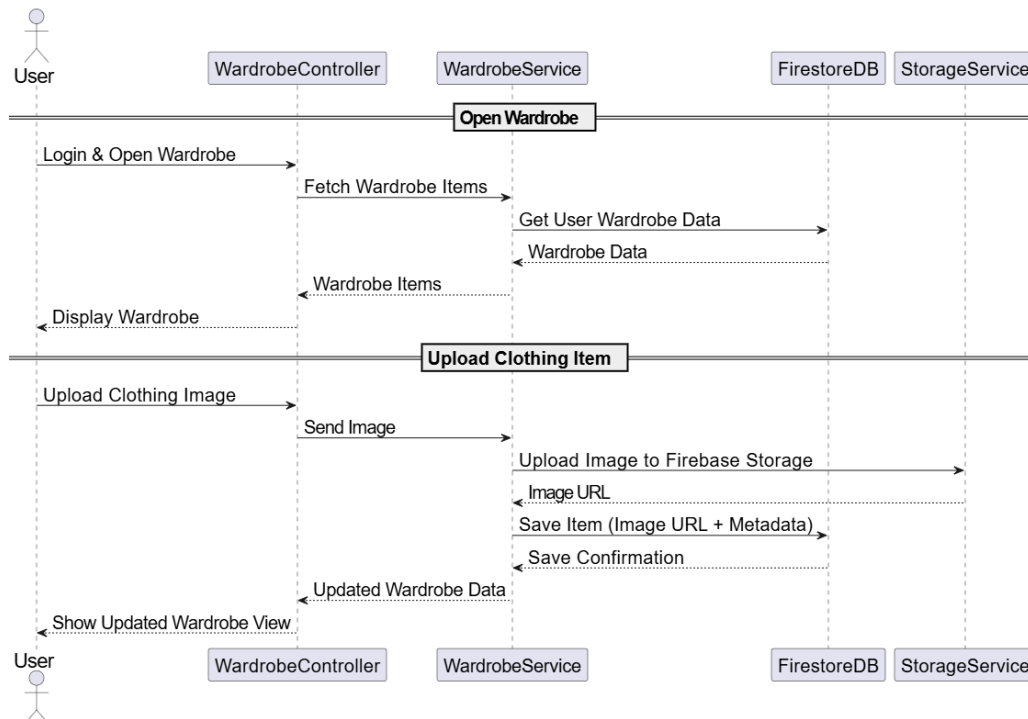


Figure 4.10: Sequence diagram for Wardrobe Management

4.3.1.3. Sequence diagram for Virtual Try-on

The Figure 4.11 illustrates how the user interacts with the virtual try-on feature [13]. The user opens the try-on module and selects a clothing item from their wardrobe. The app fetches the item image and overlays it onto the user's live camera view. The user can adjust the fit, preview how the outfit looks, and take a snapshot.

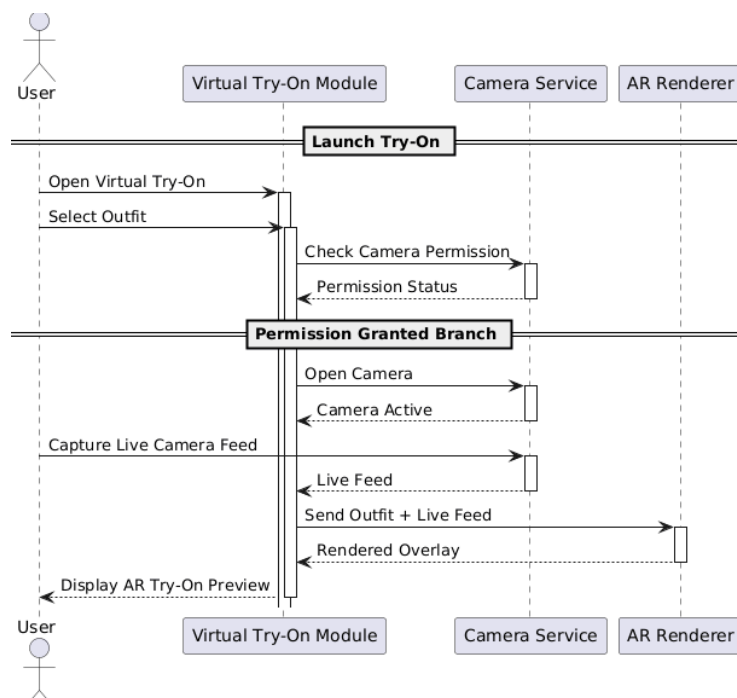


Figure 4.11: Sequence diagram for Virtual Try-on

4.3.1.4. Sequence diagram for AI-Based Outfit Suggestions

The Figure 4.12 illustrates how the user selects either clothing items, an occasion, or a preferred style, and the system processes this input to generate outfit suggestions. The wardrobe database provides item details, and the recommendation engine analyses colours, styles, and trends. The AI then sends back personalized outfit combinations, which the user can view, save, or regenerate.

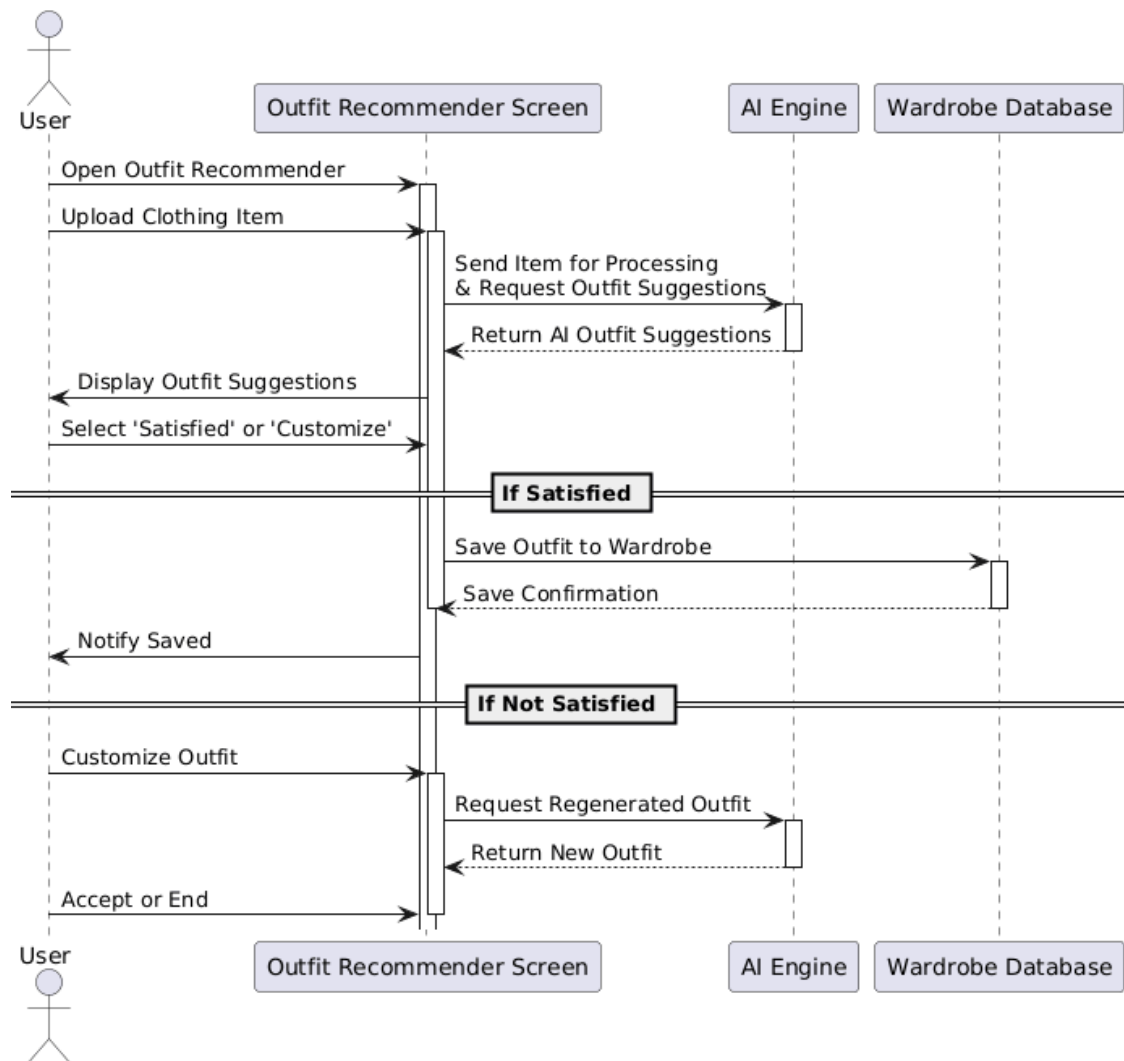


Figure 4.12: Sequence diagram for AI-Based Outfit Suggestions

4.3.1.5. Sequence diagram for Smart Shopping

The Figure 4.13 illustrates how users interact with the Smart Shopping module. The user selects a clothing category such as “Eastern Wear” or “Shirts.” The system retrieves and displays a list of relevant brands offering items in that category [12]. When the user selects a brand, the app loads the brand details and redirects the user to the official website or store page for shopping.

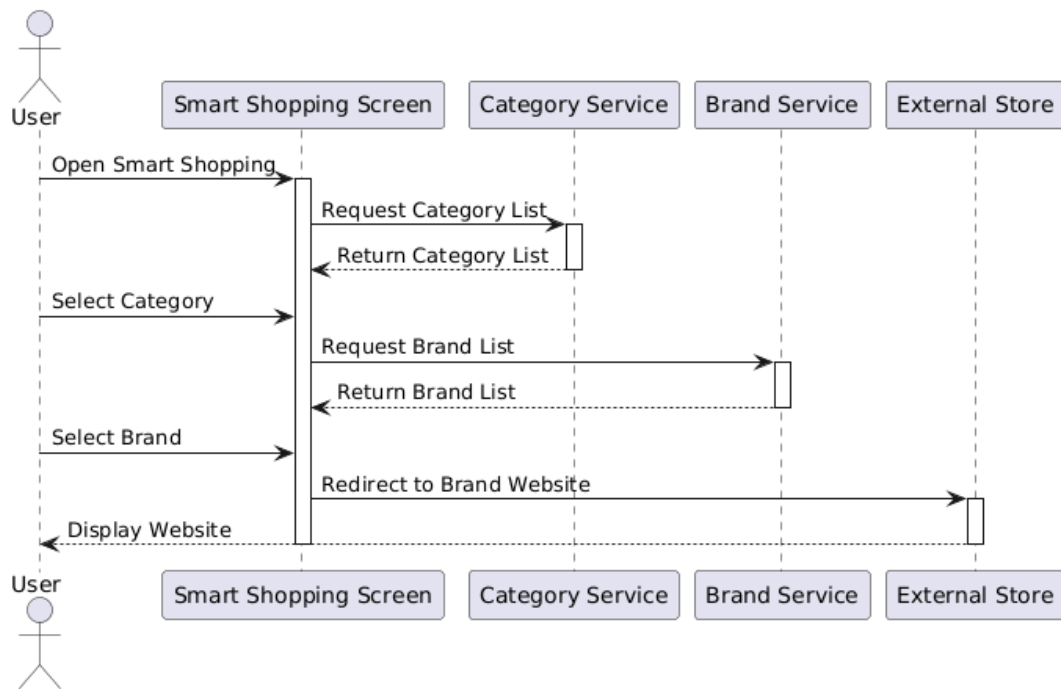


Figure 4.13: Sequence diagram for Smart Shopping

4.3.1.6. Sequence diagram for AI-Based Fashion Assistant

The Figure 4.14 illustrates how the user sends a text or voice query, the AI processes it, generates a relevant fashion response, and returns the answer in text, audio, or image form. This flow illustrates quick and personalized fashion guidance from the AI assistant.

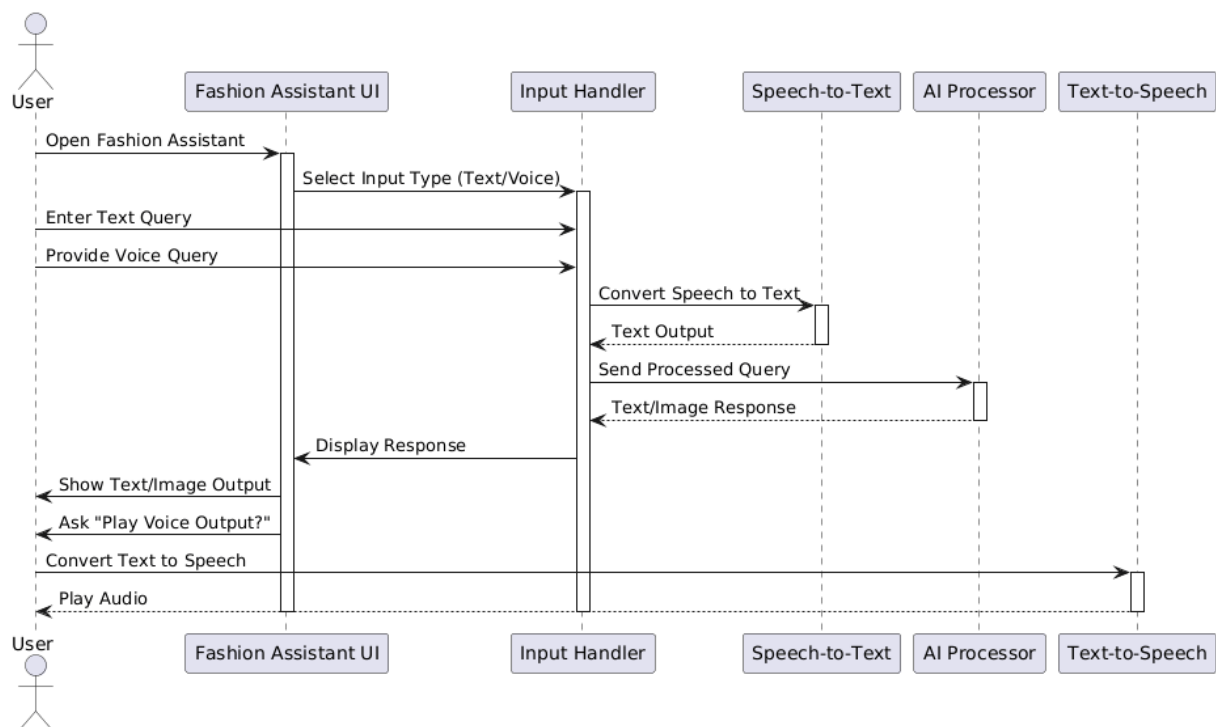


Figure 4.14: Sequence diagram for AI-Based Fashion Assistant

4.3.1.7. Sequence diagram for Fashion Content

The Figure 4.15 illustrates how users interact with the Fashion Content module. The user opens the Content Hub, and the system loads articles from the database. When the user selects an article, the system displays its full content [6]. The user can like, comment, or save the article, and the system updates the interaction records. Writers upload articles, which are reviewed and approved by the admin before being published to the Content Hub.

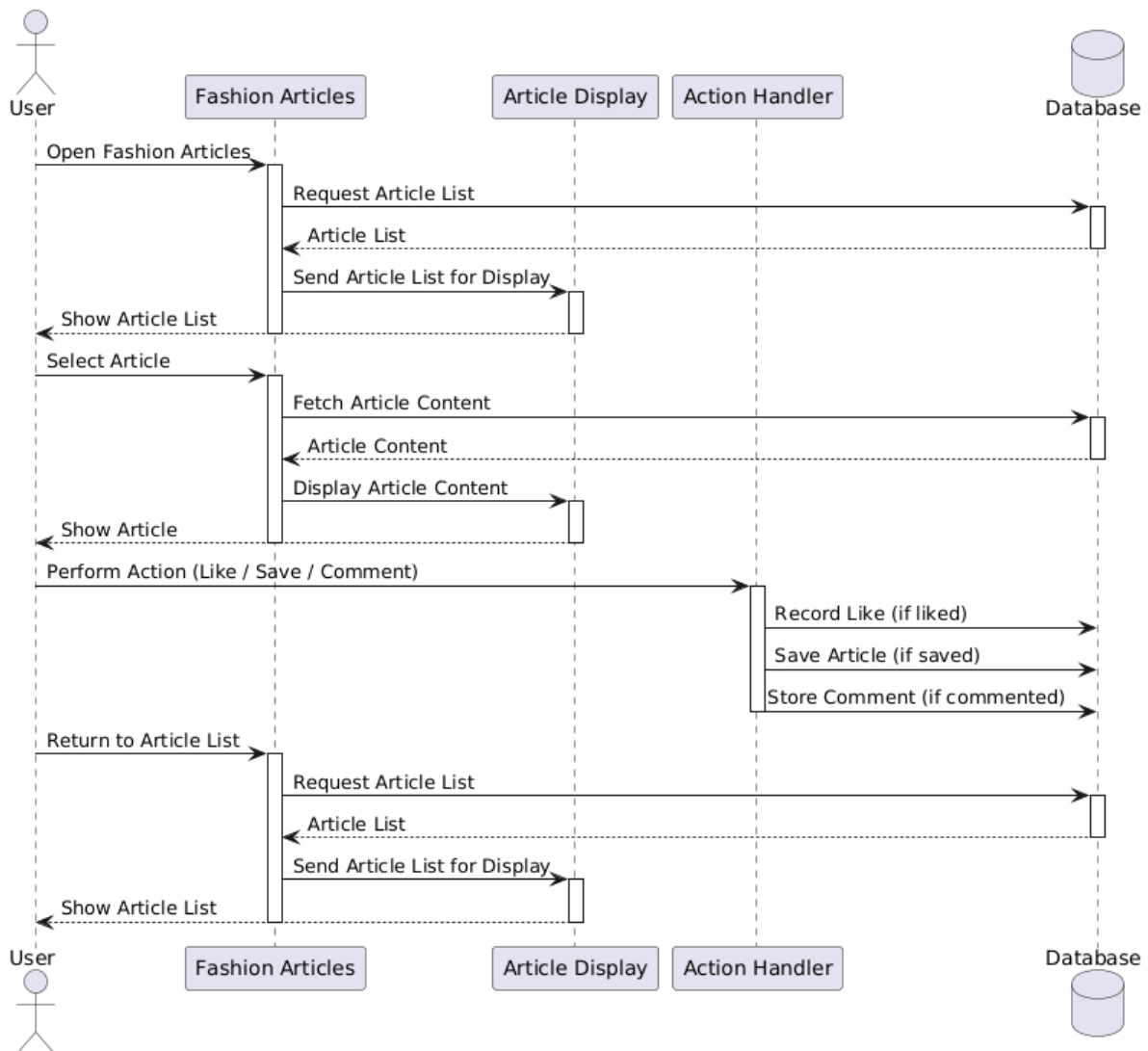


Figure 4.15: Sequence diagram for Fashion Content

4.3.1.8. Sequence diagram for Admin

The Figure 4.16 illustrates Admin is responsible for managing the platform's content and users. The admin can review, approve, or reject submitted content by writers and manage user accounts (e.g., activate, deactivate, or update user details). The sequence diagram illustrates

the interactions between the admin, the system, content submissions, and the user management module to perform these tasks efficiently.

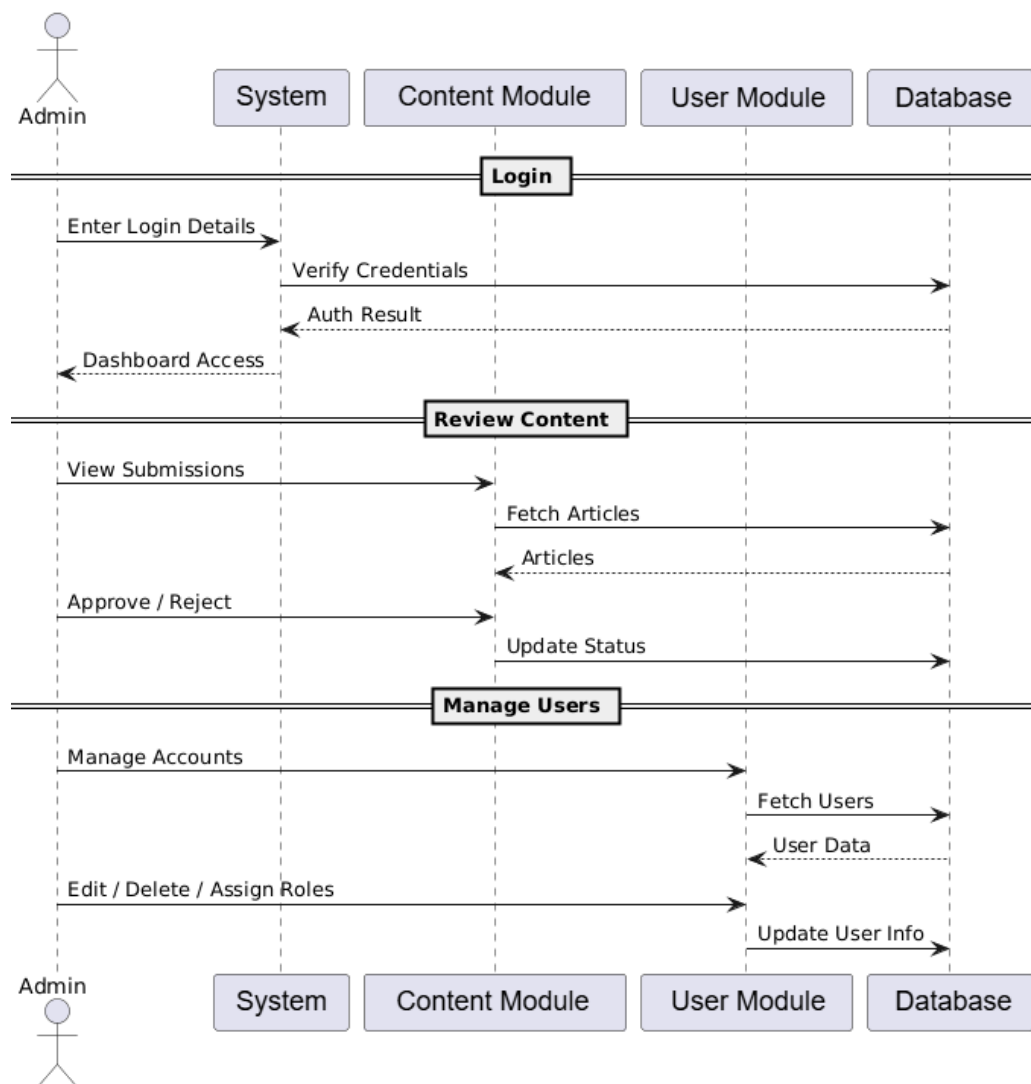


Figure 4.16: Sequence diagram for Admin

4.3.1.9. Sequence diagram for Writer

The Figure 4.17 illustrates the workflow of a writer within the Outfitly platform. The writer starts by creating and uploading an article through the app interface. Once submitted, the article is sent to the admin for review. The admin examines the content for quality, relevance, and compliance with platform guidelines. If the article meets all criteria, the admin approves it, and the content is then published to the Content Hub for all users to access. In case of revisions, the admin can send feedback back to the writer, who can edit and resubmit the article. This process ensures that only high-quality, curated content is made available on the platform, maintaining the integrity and value of the Content Hub.

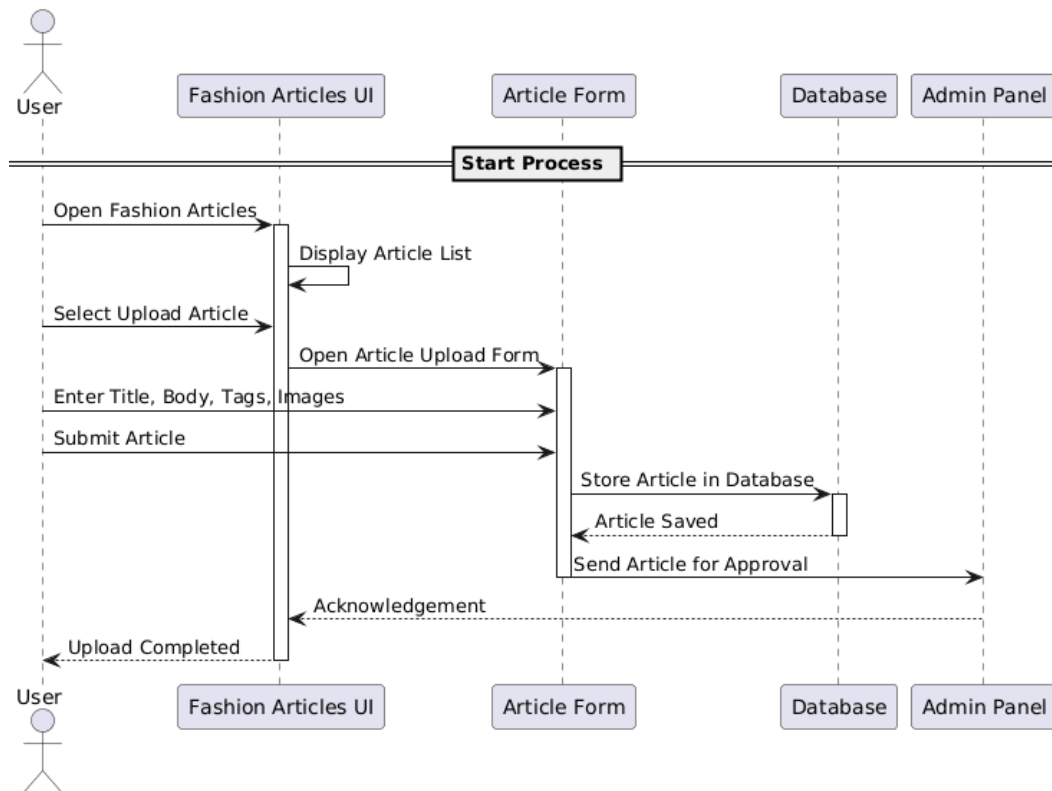


Figure 4.17: Sequence diagram for Writer

4.4. Class Diagram

A class diagram is a static structure diagram in UML that represents the structure of a system by showing its classes, attributes, methods, and relationships. It is widely used in object-oriented systems to visualize, document, and construct executable code. Class diagrams describe the static view of an application and are crucial for modelling system functionalities and collaborations.

4.4.1. Class Diagram for Outfitly

The Figure 4.18 illustrates the class diagram provides a comprehensive and well-structured representation of the system's object-oriented architecture, capturing the key components and their interactions within the AI-powered wardrobe and outfit planning platform. At the heart of the system is the User class, which serves as the primary actor and central point of interaction. It contains essential attributes such as userID, name, email, password, and role, along with crucial methods for user authentication, managing the personal wardrobe, interacting with AI-based fashion recommendations, and updating personal details. The User class maintains relationships with multiple modules, ensuring that each user's experience is personalized and seamless.

The ClothingItem class represents individual wardrobe items and stores attributes like category, color, fabric, season, brand, and imageURL. Users can add, edit, categorize, and organize these items to maintain a digital wardrobe. The ClothingItem class interfaces with the AIOutfitRecommendation module, which leverages attributes of clothing items, color harmonization rules, style compatibility, and user preferences to generate curated outfit suggestions. This AI module works closely with helper modules such as WeatherGuide (which advises on weather-appropriate clothing), FashionAssistant (which provides AI-driven styling tips), and VirtualTryOn (which allows users to visualize combinations of clothing items in a realistic, interactive manner).

The platform also incorporates a content management system to deliver high-quality fashion content. The Writer class represents content creators who can submit, edit, or remove fashion-related articles. These articles are represented by the FashionPost class, which includes postID, title, description, image, tags, and timestamps for creation and modification. Users can engage with content via the Comment class, providing feedback, suggestions, and interactions with the published posts, enhancing user engagement and community participation.

Administrative control is handled by the Admin class, which has elevated privileges to manage users, monitor system activity, oversee fashion posts, approve or reject AI-generated recommendations, and maintain the overall integrity of the platform. The Admin ensures that both user activity and content adhere to platform standards, while also enabling moderation of comments and posts to prevent misuse or inappropriate content.

Additionally, the system design emphasizes modularity, scalability, and separation of concerns. Each class encapsulates specific responsibilities, reducing interdependencies and promoting maintainability. The architecture supports future extensibility, such as integrating new AI models, adding additional wardrobe analytics, or expanding content types. Relationships between classes, such as User–ClothingItem, User–FashionPost, and AIOutfitRecommendation–ClothingItem, clearly define the data flow and interactions in the system.

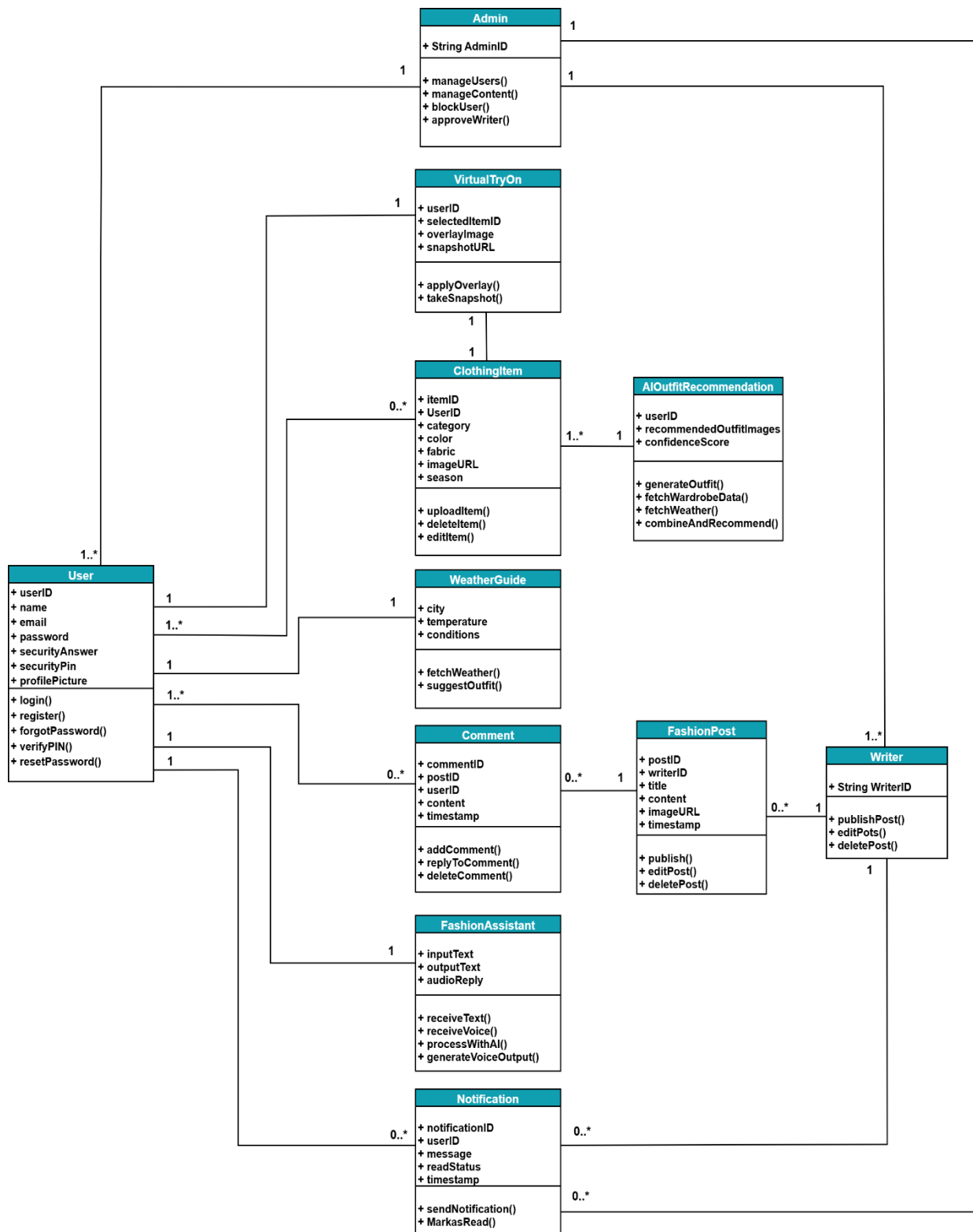


Figure 4.18: Class Diagram

Chapter 5

IMPLEMENTATION

5. Implementation

The Outfitly is implemented as a Flutter mobile application with Firebase backend. Firestore handles real-time data storage and synchronization, while Firebase Authentication manages secure login and registration. The app integrates external APIs like OpenWeather for weather-based suggestions, Hugging Face for AI-driven outfit recommendations, and Vercel Analytics for user behaviour tracking. Each module wardrobe digitization, outfit recommendation, weather-based styling, and user interaction is developed as an independent component and integrated via service controllers for reusability and optimized performance.

5.1. Algorithm

This section outlines the key algorithms and decision logic implemented in the major modules of the wardrobe system. These algorithms manage user profiles, wardrobe image processing, AI-based outfit recommendations, and real-time weather adaptation.

5.1.1. User Authentication

Function: Handles secure login, registration, and session management using Firebase Authentication [14].

Input: email, password OR Google Auth.

Output: Login success, session token, or new user creation.

Pseudocode:

1. BEGIN
2. IF user selects "Login with Email/Google" THEN
3. Authenticate using FirebaseAuth
4. IF account exists THEN
5. Grant access and retrieve user profile from Firestore
6. ELSE
7. Create new profile in Firestore
8. ENDIF
9. END

5.1.2. Wardrobe Upload and Digitization

Function: Allows user to upload clothing images and assign attributes such as type, colour, and fabric.

Input: Clothing Image.

Output: Categorized wardrobe item stored in Firestore.

Pseudocode:

1. BEGIN
2. User uploads clothing image
3. Display form for user to select:
 - a) Clothing type (e.g., shirt, pants, dress)
 - b) Fabric/material
 - c) Colours
4. Extract user-selected attributes
5. Store item in Firestore under userID
6. END

5.1.3. AI Outfit Recommendation

Function: Generates personalized outfit suggestions by matching uploaded wardrobe items based on style, colour, and weather.

Input: userID, wardrobe items (shirts, pants, optional shoes)

Output: Recommended outfit with a matching score

Pseudocode:

1. BEGIN
2. User uploads shirt, pants, and optional shoe images
3. Store images temporarily in app memory
4. When user clicks "Generate Outfit":
5. Send payload directly to Hugging Face API
6. Receive outfit recommendation + matching score
7. Display recommended outfit to user
8. END

5.1.4. Shopping Integration Module

Function: Enables users to browse and purchase clothing items from external fashion stores when suitable outfit options are not available in their wardrobe.

Input: User-selected category, weather/occasion trigger, or missing wardrobe item.

Output: Recommended shopping items with product details, price, and direct purchase link via API.

Pseudocode:

1. BEGIN
2. Display categories (e.g., Eastern wear, Shoes).
3. User selects a category
4. Display list of brands related to selected category
5. User clicks a brand
6. Show brand description + product type
7. IF user clicks "Buy Now" THEN
8. Redirect to brand's official website link
9. ENDIF
10. END

5.1.5. Weather-Based Outfit Recommendation

Function: Suggests suitable attire according to real-time weather conditions.

Input: User location (city), weather data (temperature, humidity, condition)

Output: Outfit suggestion (warm, casual, summer wear, etc.)

Pseudocode:

1. BEGIN
2. Request city/location from user profile
3. Call OpenWeather API for temperature, humidity, and condition
4. IF temperature < threshold THEN
5. Prioritize warm clothing
6. ELSE
7. Suggest light outfits
8. ENDIF
9. END

5.1.6. Skin Tone Detection and Suitable Colour Suggestion

Function: Detects user skin tone from uploaded images via Hugging Face API and suggests matching clothing colours.

Input: User image of face/skin region

Output: Detected skin tone category and recommended clothing colours.

Pseudocode:

1. BEGIN
2. User uploads a clear image of face/skin
3. Preprocess the image, crop region of interest (face/skin area)
4. Send pre-processed image to Hugging Face API endpoint hosting CNN skin tone model
5. Receive prediction: skin tone category (e.g., Light, Mid-Dark, Dark)
6. Map predicted skin tone to a predefined colour palette
7. Display recommended clothing colours to user
8. END

5.1.7. Fashion Content Module

Function: Delivers approved fashion-related articles such as fabric care guides, colour palette tips, seasonal trends, and styling suggestions. Users can browse articles, read full content, like posts, and leave comments.

Input: User preferences, clothing attributes, or selected garment

Output: Display of relevant content cards

Pseudocode:

1. BEGIN
2. Fetch approved articles from Firestore (status = approved)
3. Display list of articles (title, image, short description)
4. User selects an article
5. Display full article content
6. IF user clicks "Like" THEN
7. Update like count in Firestore
8. ENDIF
9. IF user submits a comment, THEN
10. Save comment in Firestore under article ID
11. ENDIF
12. END

5.1.8. Virtual Try-on

Function: Users can virtually try on outfits by overlaying dress images over live camera feed and adjusts the dress position manually using drag/resize gestures and can capture a snapshot.

Input: User-selected dress image and live camera feed from device.

Output: Preview image with the dress overlay and optionally save final try-on snapshot to user gallery or app storage

Pseudocode:

1. BEGIN
2. User opens Virtual Try-On feature
3. Activate device camera in live preview mode
4. Display a UI prompt for user to select a dress image from wardrobe
5. Overlay selected dress image on top of live camera feed (default centred)
6. Enable user gestures:
 - a. Drag to reposition dress
 - b. Pinch to zoom in/out
 - c. Rotate gesture for angle adjustment (optional)
7. User adjusts overlay manually to align with their body
8. On user action "Capture":
 - a. Take snapshot of camera feed with current overlay position
 - b. Save snapshot to local device or Firestore under userID (optional)
9. Display confirmation message with options: Save and Retake
10. END

5.1.9. AI Fashion Chatbot

Function: Provides interactive conversation-based recommendations, answers fashion queries, and helps users decide what to wear.

Input: User message or voice query

Output: AI-generated response using Hugging Face conversational model

Pseudocode:

1. BEGIN
2. User sends message to chatbot interface
3. Send message to Hugging Face conversational API
4. Receive AI-generated response
5. Display response in chat UI
6. IF response requires wardrobe access, THEN
7. Retrieve wardrobe data from Firestore
8. Provide personalized outfit suggestion
9. ENDIF
10. END

5.2. External APIs

The Table 5.1 shows that Outfitly uses multiple external APIs to enhance user experience, automate fashion-related insights, and ensure reliable and secure functionality. These APIs enable features such as AI-based clothing recognition, virtual try-on capabilities, cloud-based data storage, user authentication, and real-time styling suggestions. By integrating these APIs, the application ensures scalability, personalization, and a seamless user interface that mimics professional styling platforms. This integration empowers users with intelligent wardrobe management, providing accurate, real-time recommendations tailored to their preferences and lifestyle.

Table 5.1: External API's

API Name	Description	Purpose	Function/Class Used
Firebase Auth	Authentication service	Secure login, session control	AuthService
Firestore Database	Cloud NoSQL storage	Wardrobe, user preferences, content storage	FirestoreService
OpenWeather API	Real-time weather data	Weather-based outfit recommendation	WeatherService

Hugging Face API (Recommendation Model)	Machine learning inference	AI outfit prediction	AIRecommendationService
Hugging Face Chat API	Conversational AI	AI stylist chatbot interaction	AIChatbotService
Hugging Face Skin Tone Detection API	CNN-based skin tone classification API	Detects user skin tone from images and suggests suitable clothing colours	SkinToneService
Vercel Analytics API	Usage analytics	Track user behaviour for personalized AI model	AnalyticsService

5.3. User Interface

The AI-Powered Digital Wardrobe and Styling Application features a modern, visually appealing, and highly intuitive user interface (UI) designed to provide a seamless fashion experience. The UI follows a minimalistic, user-centred design approach to ensure that users of all ages and technical backgrounds can easily navigate and interact with the application.

Here are the primary interface designs of the AI-Powered Digital Wardrobe and Outfit Planner Application:

5.3.1. Home Screen

Figure 5.1 illustrates the Home Screen of Outfitly, which serves as the central entry point for users to access the AI-powered wardrobe and styling experience. The layout is designed to clearly communicate the app's purpose of smart wardrobe management and AI-based outfit planning, encouraging users to begin their personalized styling journey right from the first interaction. In addition to the main interface, the Home Screen provides options for both Users and Content Writers to navigate to their respective login screens, ensuring role-based access and personalized experiences.

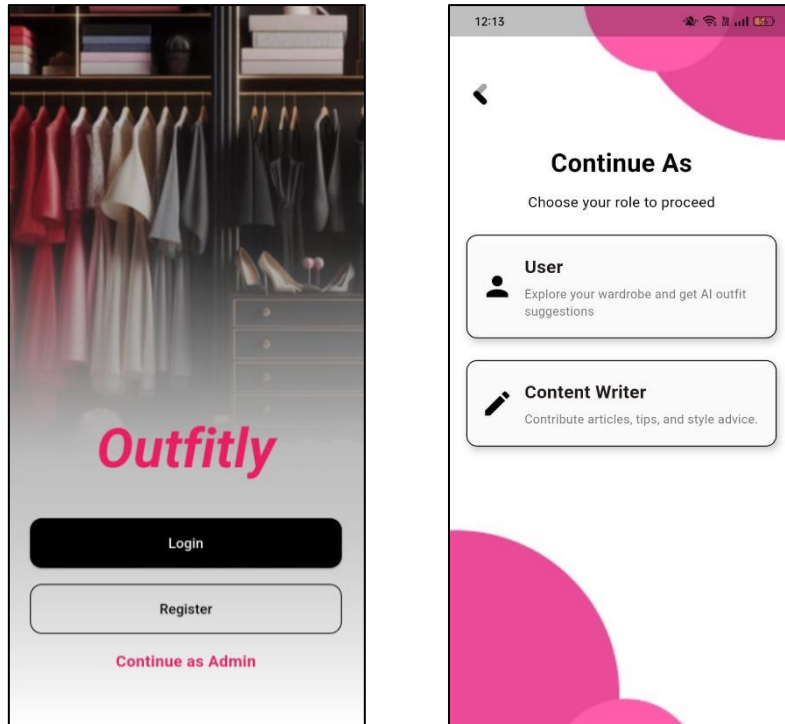


Figure 5.1: Home Screen

5.3.2. User Login Form

Figure 5.2 illustrates the login screen that shows a modern, fashion-inspired screen. A stylish card with rounded corners and soft shadows holds the email and password fields. User can also login via Google allows quick and convenient authentication.

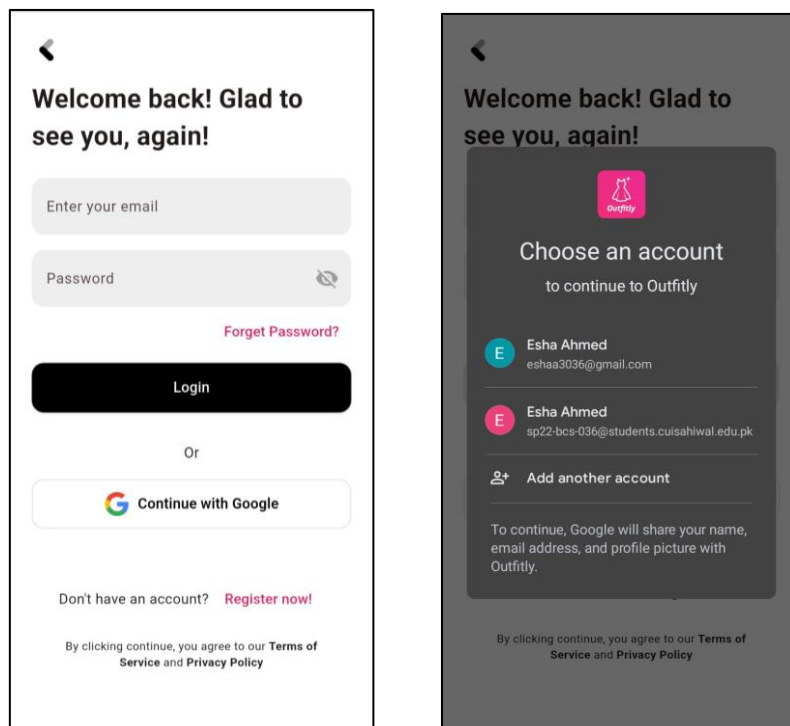


Figure 5.2: Login Form

5.3.3. Register Form

Figure 5.3 illustrates the Outfitly registration screen, designed with a clean, user-friendly interface that reflects the app's modern fashion theme. The registration card features rounded edges and soft shadows, creating an inviting and approachable aesthetic that encourages new users to join effortlessly. Users can enter their details, select their role as either a User or a Writer, and set a secure password. Additionally, the form includes options for security verification, allowing users to choose either a PIN or a security question and answer, which can be used to recover their account in case they forget their password.

The figure displays two mobile app screens for the 'Create Account' registration process. Both screens have a black header with a back arrow and the title 'Create Account'. The left screen shows a white registration card with a profile picture placeholder (a grey circle with a person icon and a camera icon), followed by input fields for Name, Email, Birthday, a dropdown for 'Select Role', a dropdown for 'Select Security Option', and a Password field. The right screen continues the form with input fields for Email, Birthday, a dropdown for 'Select Role', a dropdown for 'Select Security Option', a Password field, and a Confirm Password field. Below these is a checkbox labeled 'Agree with Terms & Conditions' (where 'Terms & Conditions' is in pink). At the bottom of the right screen is a prominent red button labeled 'Register'.

Figure 5.3: Registration Form

5.3.4. Forgot Password Screen

Figure 5.4 illustrates the forgot password screen, designed to help users securely reset their account password in a simple and intuitive manner. The interface aligns with Outfitly's modern and stylish theme, maintaining consistency with other authentication screens. To verify their identity, users can either enter the previously set PIN or correctly answer their chosen security question. Once verification is successful, the system allows users to create a new password, ensuring that account recovery is both secure and user-friendly.

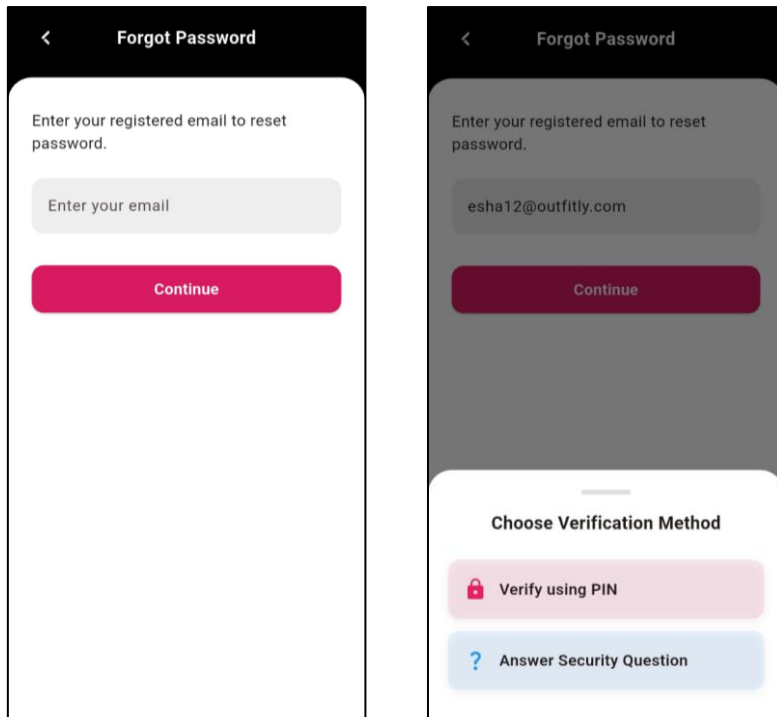


Figure 5.4: Forgot Password Screen

5.3.5. User Dashboard

Figure 5.5 shows the Outfitly app’s main dashboard, where users access their digital wardrobe, AI outfit recommendations, virtual try-on, and fashion assistant chat. It also displays recent activity, and quick access icons, offering a smooth start to personalized wardrobe planning.

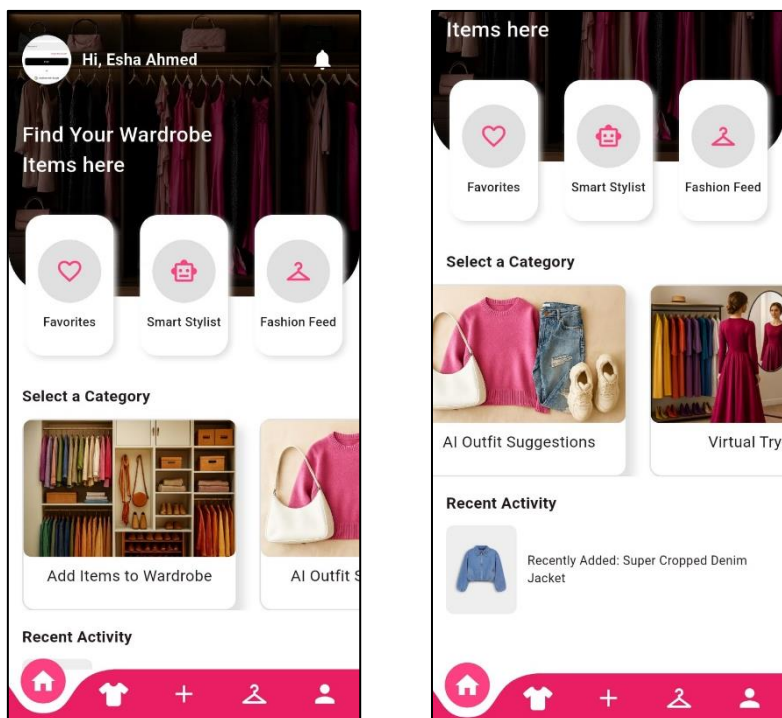


Figure 5.5: User Dashboard

5.3.6. Wardrobe Management Screen

Figure 5.6 displays the user's personal digital closet, providing a comprehensive interface to organize and manage all uploaded clothing items. Users can browse, filter, and categorize items by type, ensuring easy access to their wardrobe. Each clothing item is stored with detailed metadata, including colour, fabric, subcategory, and usage frequency. Users can view individual items in detail by clicking on them, allowing them to see all associated information. Additionally, the screen provides options to edit item details, delete items, mark favourites, or mark items as "Wear Today", giving users full control over wardrobe management.

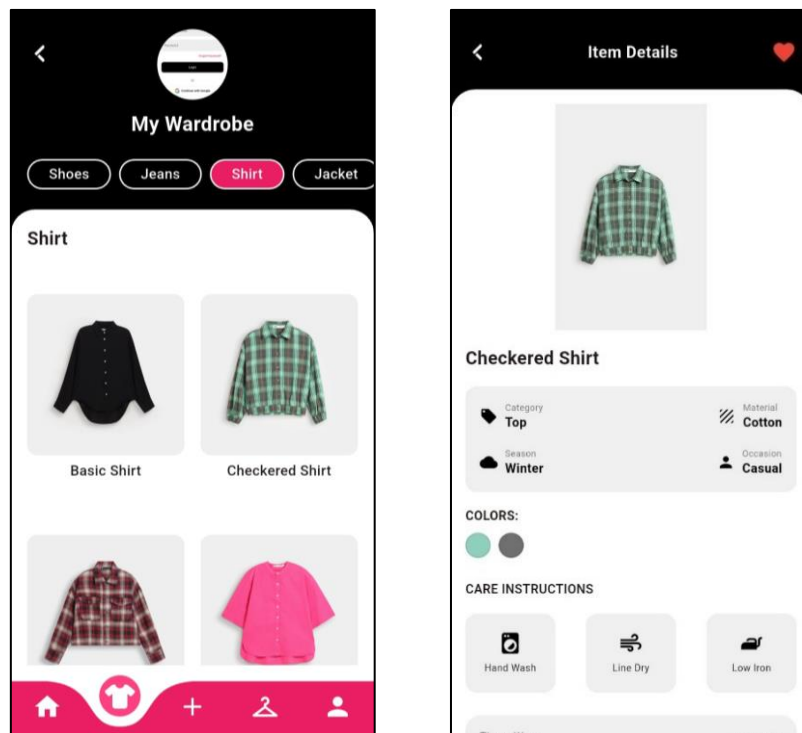


Figure 5.6: Wardrobe Management Screen

5.3.7. Add Clothing Item Screen

Figure 5.7 illustrates the add clothing item screen allows users to expand and manage their digital wardrobe by uploading new clothing items. Users can provide essential details for each item, including category (e.g., top, bottom, footwear), colour, fabric type, size, season, and any relevant tags for occasion or style. Additionally, users can upload an image of the clothing item to visually represent it in their wardrobe [3]. Users can also edit or delete existing items directly from this screen, maintaining a personalized and organized wardrobe for seamless outfit planning.

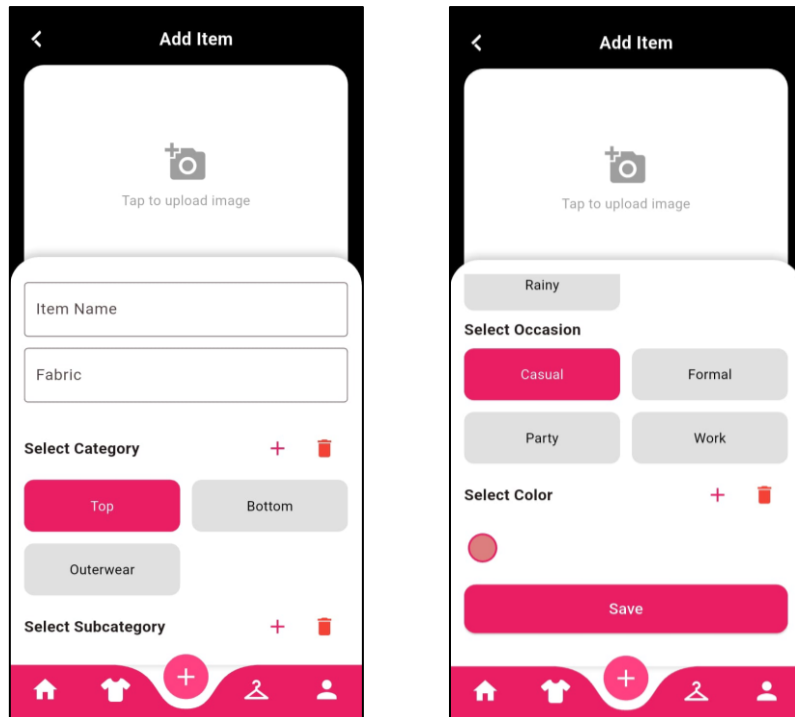


Figure 5.7: Add Clothing Item Screen

5.3.8. AI Outfit Recommendation Screen

Figure 5.8 shows AI-generated outfit suggestions tailored to the user's wardrobe, and style preferences. The system analyses uploaded clothing items to create practical and appealing outfits. Users can view item and save favourites.

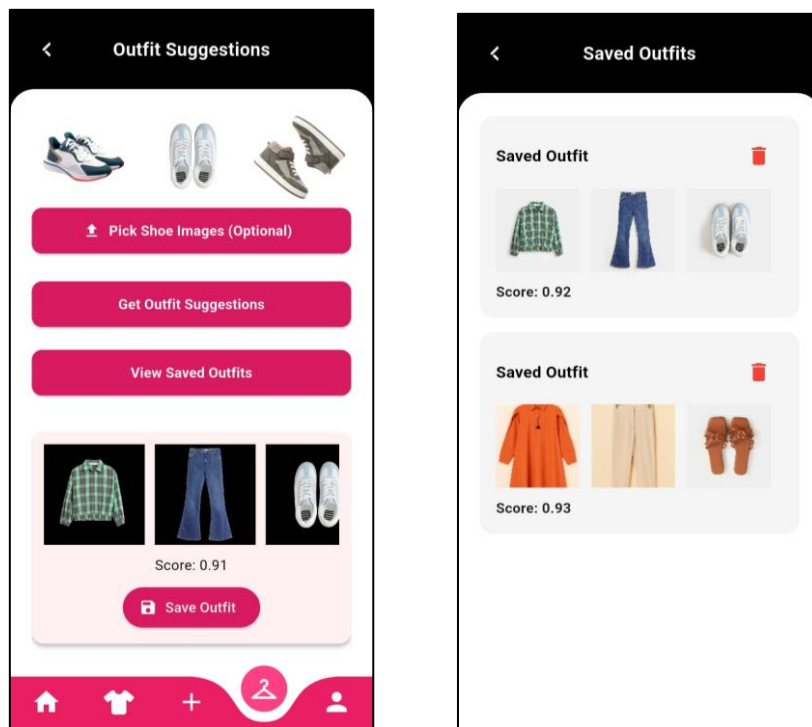


Figure 5.8: AI Outfit Recommendation Screen

5.3.9. Weather-based Outfit Suggestions Screen

Figure 5.9 The system provides weather-based outfit suggestions by automatically analyzing real-time weather data such as temperature, humidity, and forecast conditions. Based on this information, the AI recommends clothing combinations that are both comfortable and suitable for the expected weather. For example, it suggests lighter fabrics and breathable outfits for warm days, while recommending layers, jackets, or warm accessories during colder or rainy conditions. This feature ensures that users receive practical, season-appropriate outfit options without manually checking the weather.

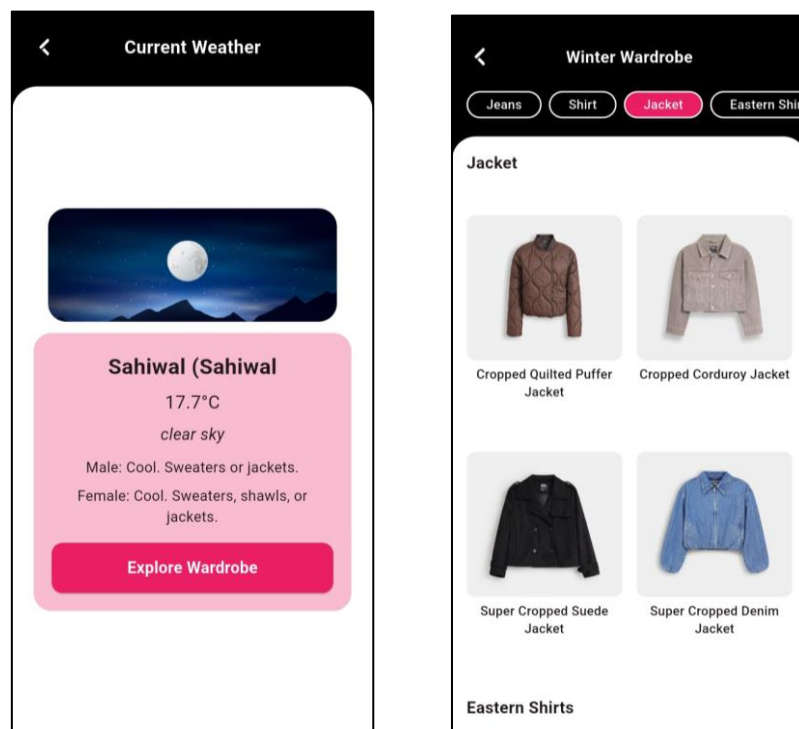


Figure 5.9: Weather-based Outfit Suggestions Screen

5.3.10. Virtual Try-On Screen

Figure 5.10 illustrates the virtual try-on screen, where users can visualize how different clothing items would look on them using either the device's camera or an uploaded image. The system overlays selected outfits onto the user's body, allowing them to adjust the position, scale, and alignment for a more accurate fit [14]. Users can experiment with multiple outfits in real time, compare looks side-by-side, and explore various styling options before making a decision. This interactive feature enhances user confidence by providing a realistic preview of outfits, supporting more informed styling and online shopping choices.

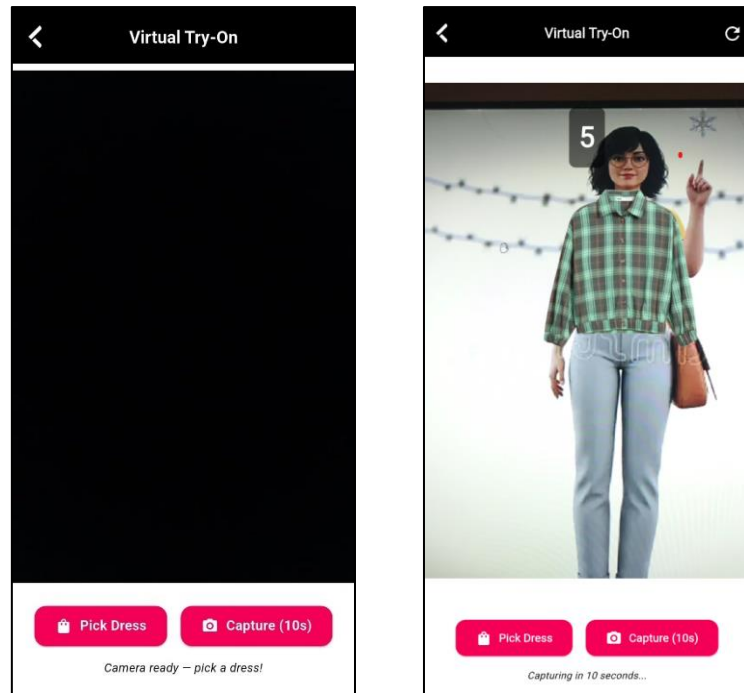


Figure 5.10: Virtual Try-on Screen

5.3.11. Outfit Planner Screen

Figure 5.11 shows the calendar-based outfit planner that helps users schedule outfits for specific days and events. It provides daily, weekly, and monthly views, allowing users to set reminders, adjust plans easily, and track previously worn items.

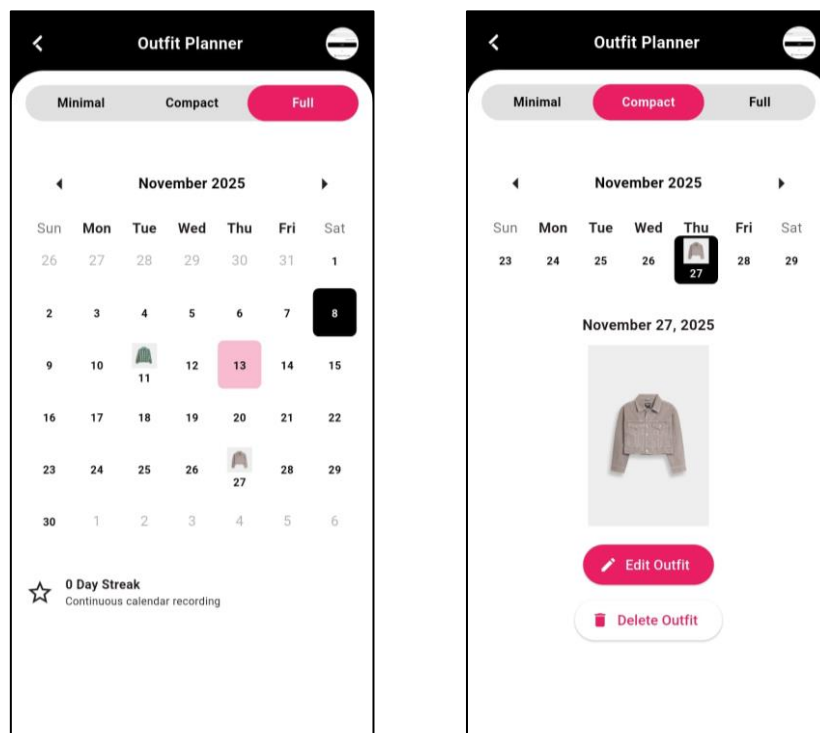


Figure 5.11: Outfit Planner Screen

5.3.12. AI Fashion Assistant Screen

Figure 5.12 illustrates the AI fashion assistant screen, which offers an interactive chat interface designed to provide users with real-time, personalized fashion guidance. Users can communicate with the assistant through text or voice input, asking questions about outfit combinations, styling tips, fabric care, and seasonal fashion recommendations [14]. The AI leverages user preferences, wardrobe data, and fashion trends to deliver tailored advice that suits individual style and occasions. Conversations are displayed in a clear, conversational layout, making the interaction intuitive, engaging, and easy to follow.

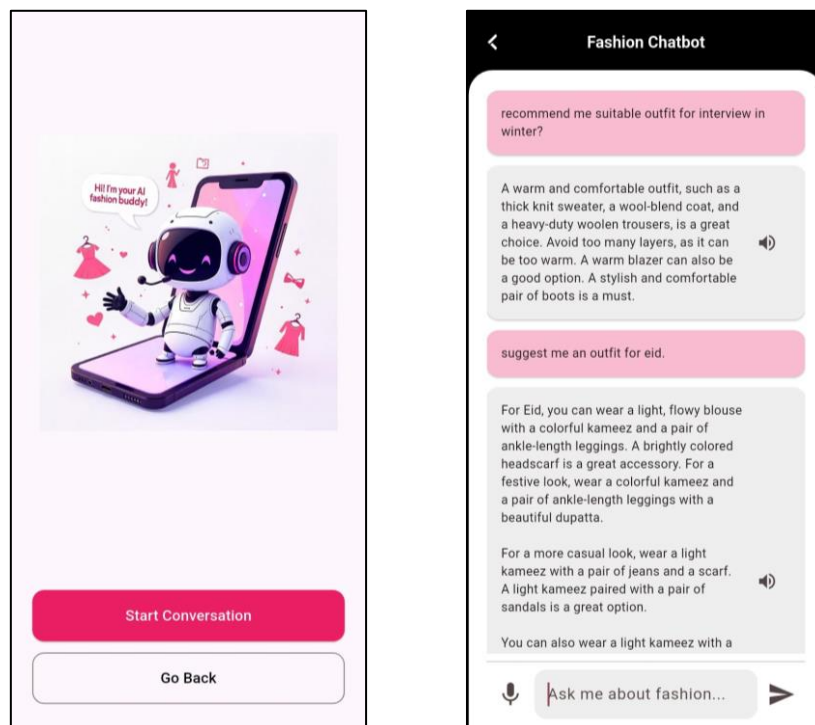


Figure 5.12: AI Fashion Assistant Screen

5.3.13. Smart Shopping Screen

Figure 5.13 displays the smart shopping module, where users can browse personalized product recommendations based on their style preferences, wardrobe gaps, and recent outfit selections. The screen presents detailed product information, including images, price, fabric, brand, and user reviews, helping users make informed purchasing decisions [15]. Users can explore trending brands, view curated collections, and compare similar items within the app. A dedicated “Buy Now” button enables seamless shopping by redirecting users to the brand’s official website, ensuring a smooth transition from browsing to purchase.

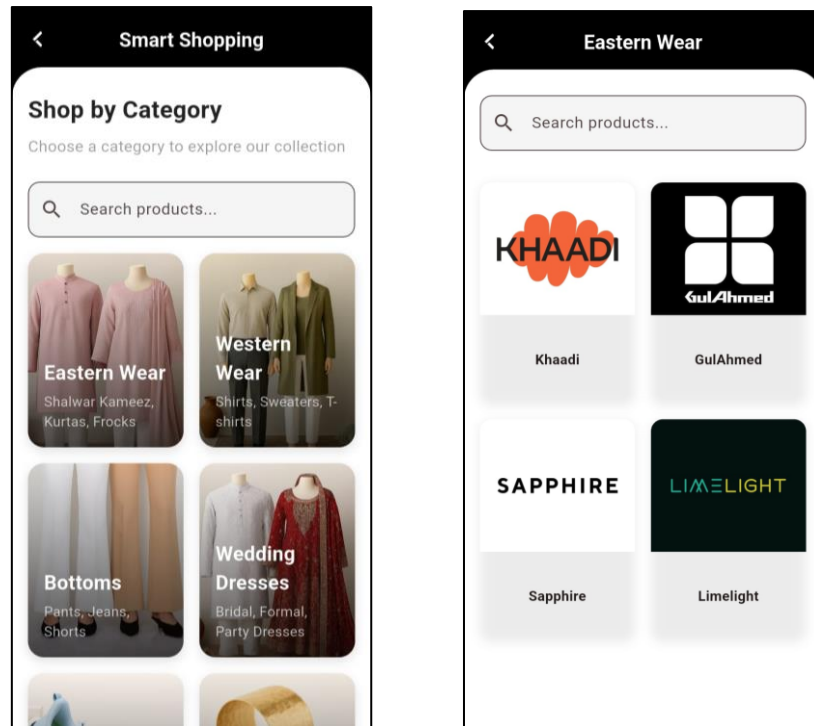


Figure 5.13: Smart Shopping Screen

5.3.14. Content Hub Screen

Figure 5.14 illustrates the content hub module, which provides users with a dynamic feed of outfit inspirations, seasonal trends, styling tips, and articles. Users can interact with posts through likes, comments, and shares, fostering engagement and community within the app.

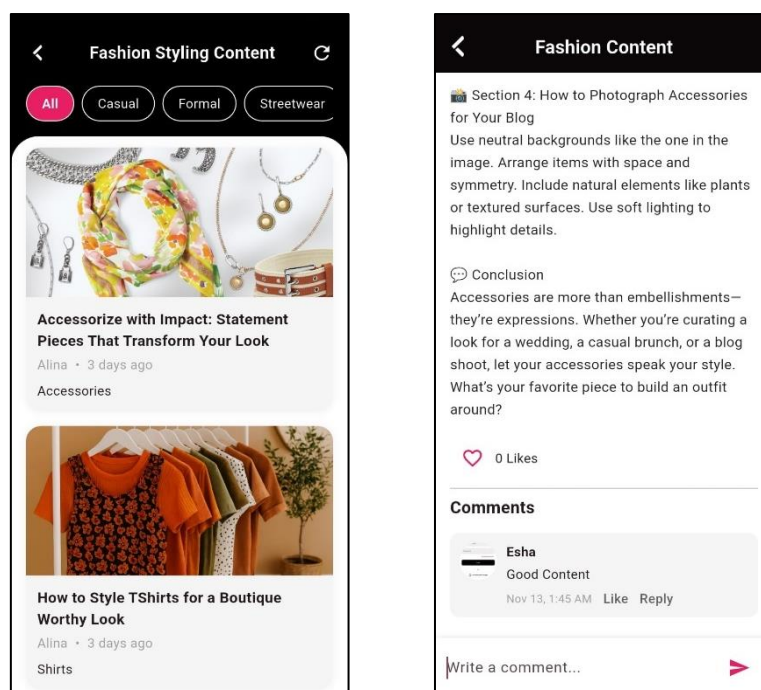


Figure 5.14: Content Hub Screen

5.3.15. Skin Tone Detection and Colour Advisor

Figure 5.15 illustrates that the Colour Palette Advisor allows users to provide an image of their skin tone, which the system analyses to recommend clothing colours that best complement their complexion [3]. The module considers factors such as undertones, contrast levels, and style preferences to generate a personalized palette of suitable colours. This helps users make informed fashion choices, coordinate outfits effectively, and enhance their overall appearance. Additionally, the advisor can suggest seasonal or occasion-specific colour combinations, making it a practical tool for wardrobe planning and styling decisions.

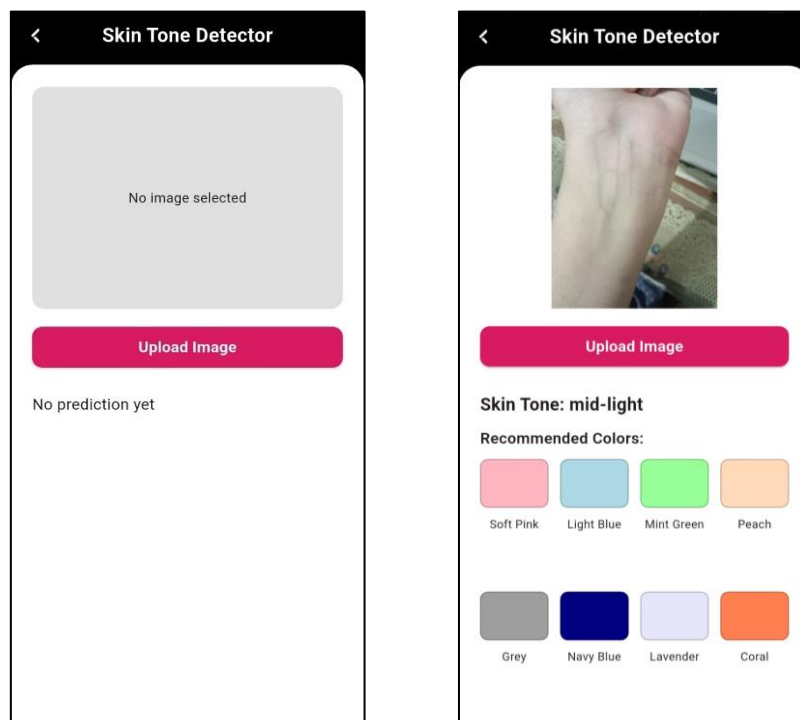


Figure 5.15: Skin Tone Detection and Colour Advisor

5.3.16. Fabric Care Advisor

Figure 5.16 illustrates that the Fabric Care Advisor provides users with comprehensive guidance on maintaining and preserving their clothing items. The module delivers step-by-step instructions for washing, drying, ironing, and proper storage based on each item's fabric type. It helps users prevent common issues such as shrinkage, colour fading, fabric damage, and texture loss. By offering tailored care recommendations, this feature supports more sustainable wardrobe management and encourages users to maintain their clothing responsibly. In addition to preventing damage, the Fabric Care Advisor promotes sustainable wardrobe habits by encouraging users to take better care of their clothes, reducing the likelihood of premature wear and replacement.

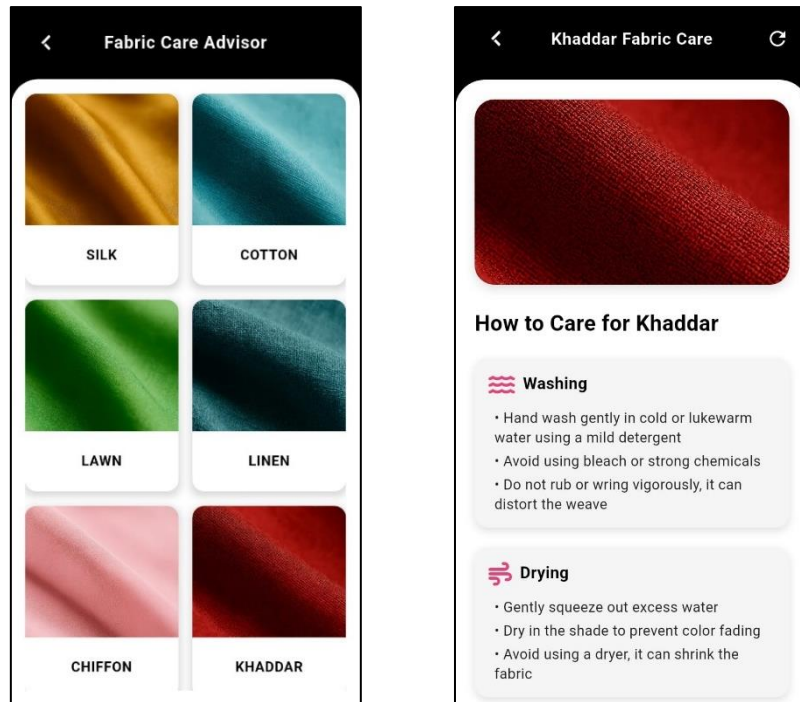


Figure 5.16: Fabric Care Advisor

5.3.17. User Profile Screen

Figure 5.17 illustrates the User Profile module, which enables users to manage and customize their personal information, preferences, and wardrobe settings within the Outfitly app. Users can view and update details such as their username, profile picture, style preferences, and location, ensuring their account reflects their current tastes and needs.

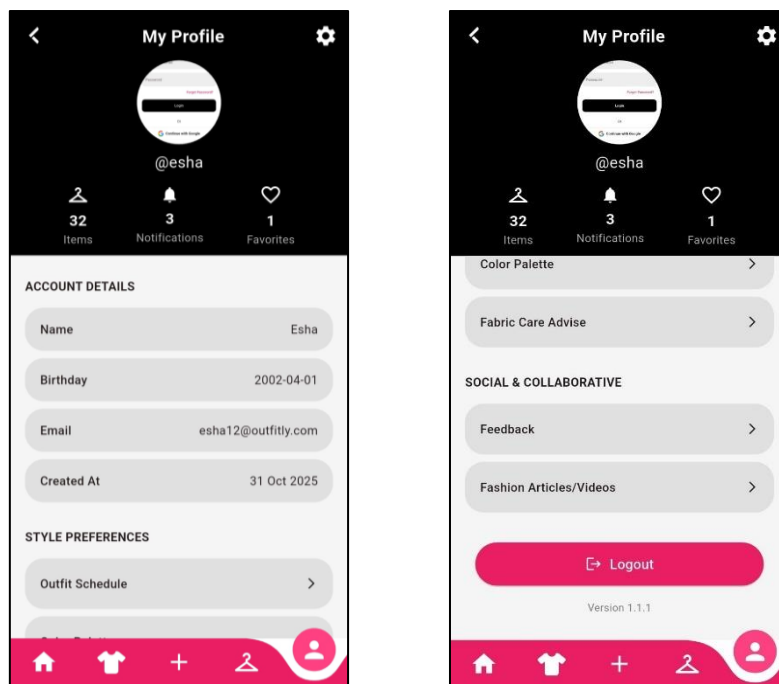


Figure 5.17: User Profile Screen

5.3.18. User Feedback Screen

Figure 5.18 illustrates the feedback screen allows users to share their thoughts, suggestions, and issues directly with the app's support team. Users can submit new feedback through a simple form and view all previously submitted feedback in an organized list. Each entry displays the user's message, submission date, and the corresponding admin response. This enables users to track the status of their feedback, review past concerns, and read replies or resolutions provided by the admin. The screen helps maintain transparent communication and enhances the overall user experience by ensuring that user voices are acknowledged and addressed.

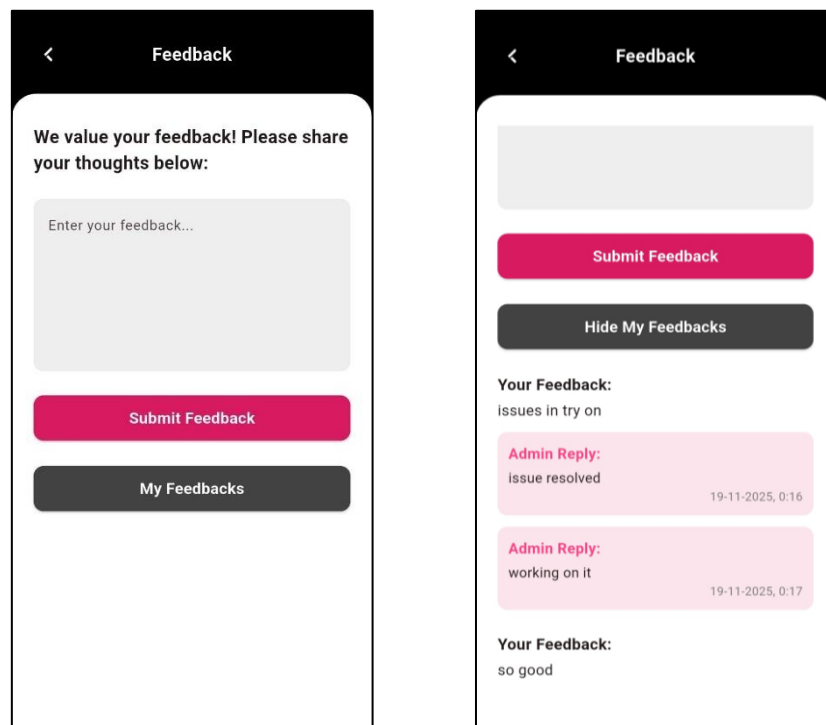


Figure 5.18: User Feedback Screen

5.3.19. User Settings Screen

Figure 5.19 illustrates the user settings screen allows users to manage their personal preferences and account information. From this screen, users can update their profile details, change their password, modify notification preferences, and adjust privacy settings. The screen also provides options to manage app permissions, clear cache, and control appearance settings such as light or dark mode. Additionally, users can review terms and policies or log out of their account. This screen ensures that users have full control over their app experience and account customization.

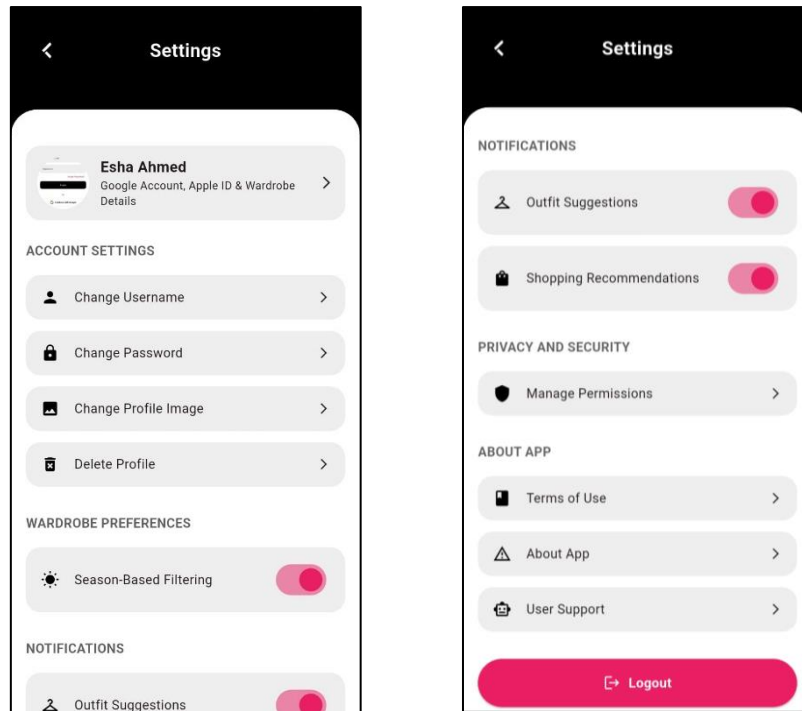


Figure 5.19: User Settings Screen

5.3.20. Writer Login and Dashboard Screen

Figure 5.20 shows the Writer Dashboard, where content creators can manage and publish fashion-related articles. It displays key metrics such as likes, comments, published posts, and pending submissions for easy monitoring. Writers can create, edit, delete, and track their articles through a secure and user-friendly interface.

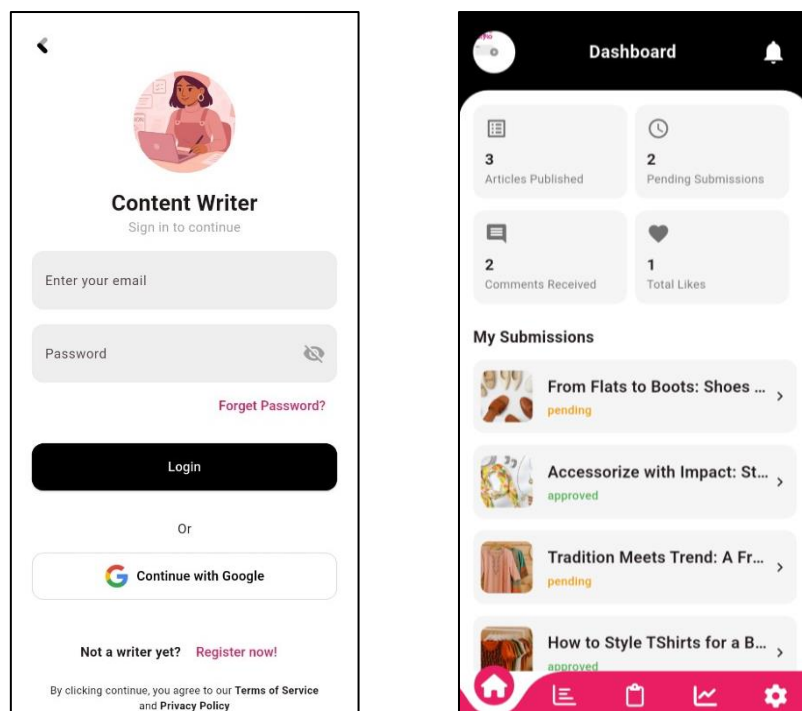


Figure 5.20: Writer Login and Dashboard

5.3.21. Write/Edit Article Screen

Figure 5.21 shows the Write/Edit Article screen, where writers can create, modify, and enhance fashion-related articles within a structured and user-friendly editing environment. The screen includes a clean and intuitive text editor that supports adding titles, inserting images, applying text formatting, and assigning relevant tags for better categorization. Writers can save their work as drafts, preview the article before publishing, and make updates at any stage of the writing process.

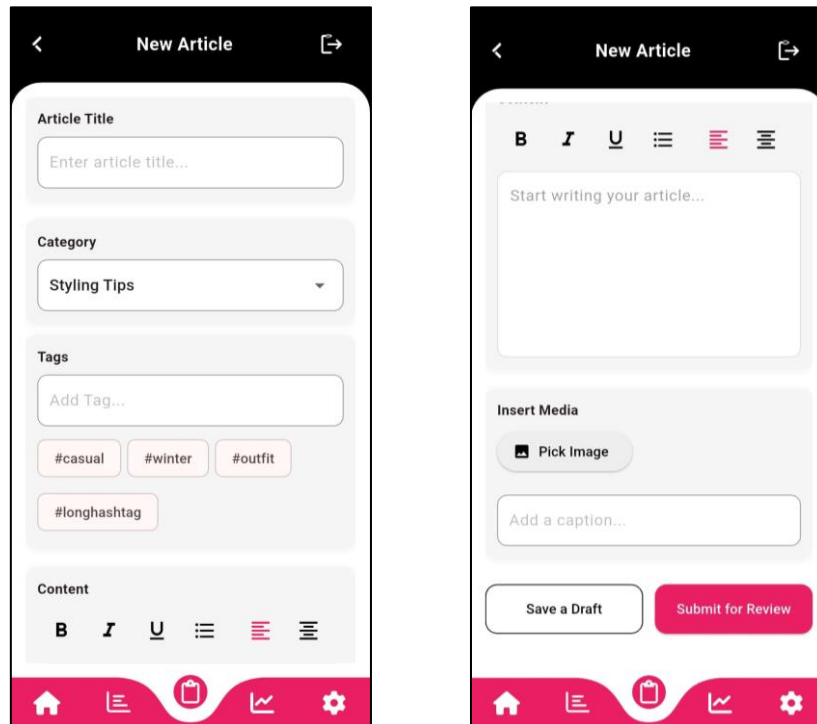


Figure 5.21: Write/Edit Article Screen

5.3.22. SEO Optimization Screen

Figure 5.22 illustrates the SEO Optimization feature, designed to help content writers enhance the online visibility of their fashion articles. The interface provides dedicated fields for entering meta titles, meta descriptions, and keyword tags, allowing writers to optimize their content for search engines effectively [5]. By integrating these SEO elements directly into the article creation process, writers can ensure that their content reaches a wider audience, attracts more traffic, and improves discoverability. Additionally, the module may offer suggestions for keyword relevance and content optimization, helping writers create high-quality, search-friendly articles that align with best SEO practices.

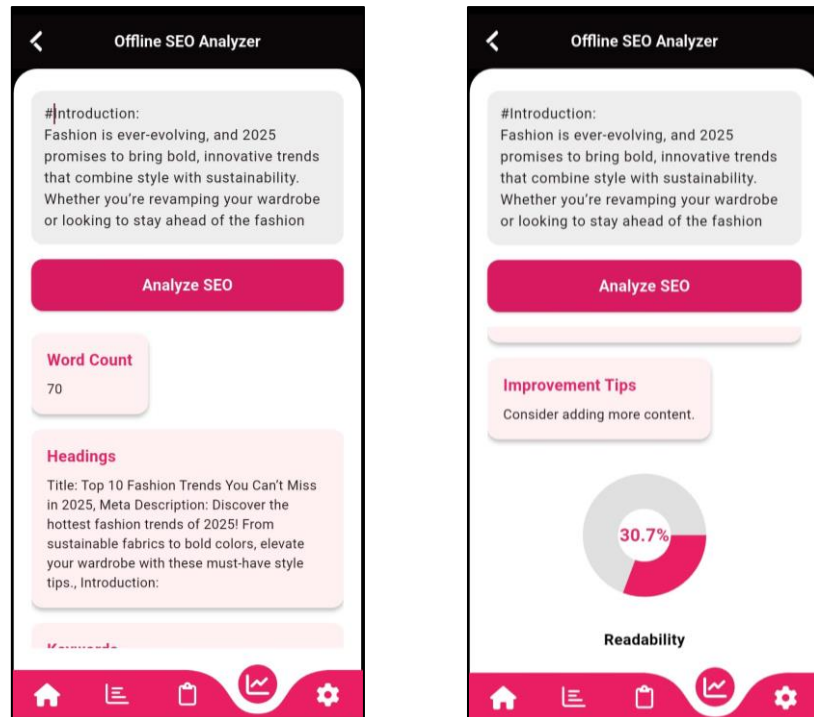


Figure 5.22: SEO Optimization Screen

5.3.23. Writer Settings Screen

Figure 5.23 illustrates the Content Writer Settings screen, which allows writers to update their profile information, change passwords, adjust account preferences, and configure privacy settings to control how their content and personal data are handled.

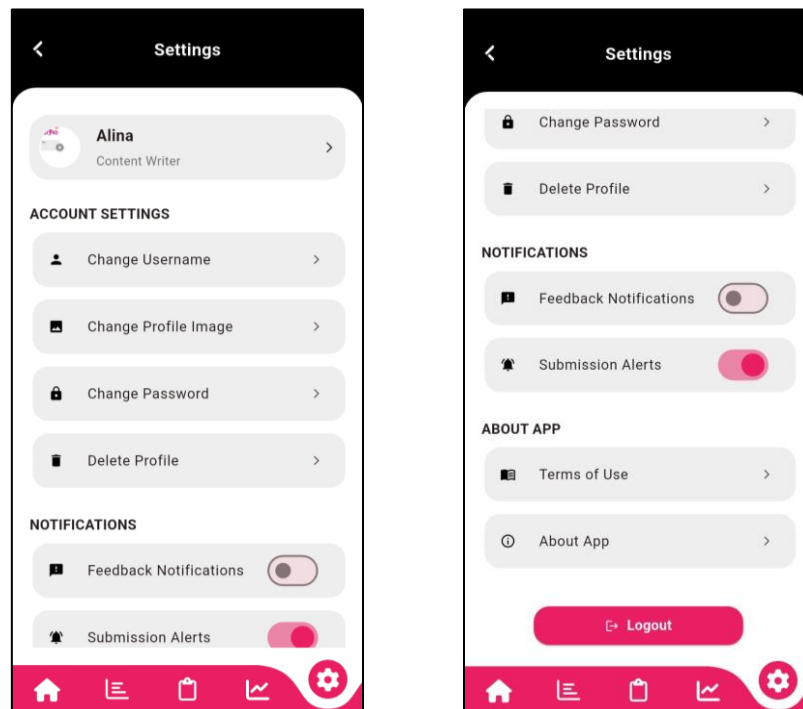


Figure 5.23: Writer Settings Screen

5.3.24. Admin Login and Dashboard Screen

Figure 5.24 illustrates the Admin Login and Dashboard screen, providing administrators with a comprehensive overview of system activity and app performance. The dashboard displays key metrics such as the number of active users, wardrobe items uploaded, published articles, AI outfit recommendations generated, and overall user engagement. Administrators can monitor user behaviour, track system usage patterns, and identify trends to make data-driven decisions for improving app functionality.

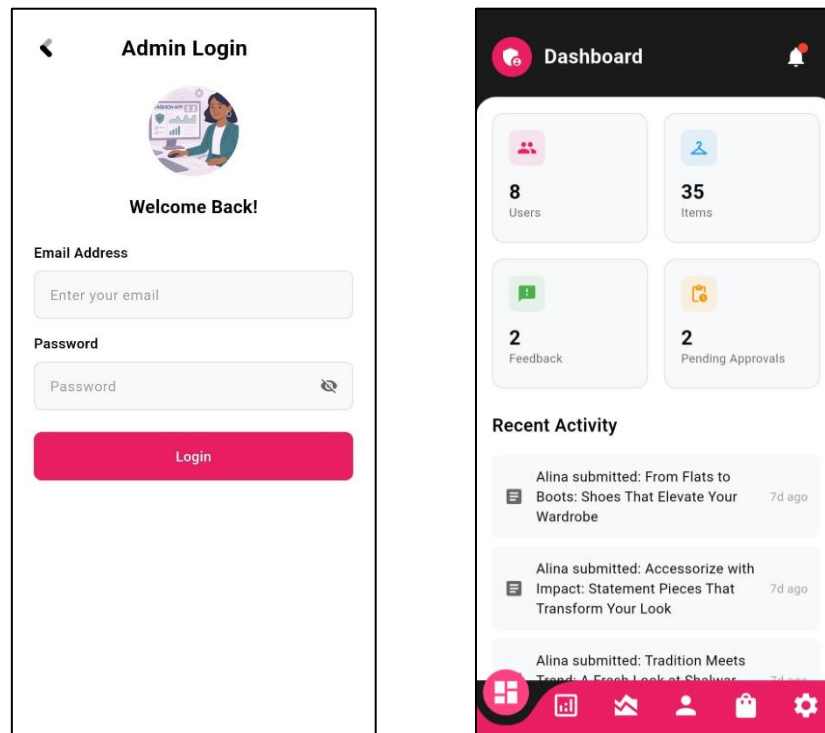


Figure 5.24: Admin Dashboard

5.3.25. User Management Screen

Figure 5.25 illustrates the User Management Screen, which provides administrators with comprehensive control over all user accounts within the Outfitly application. The screen allows admins to view detailed user profiles, edit account information, and delete accounts when necessary. The interface also enables filtering and searching of users for quick access, making it easier to manage large numbers of accounts efficiently. By providing these tools, the User Management Screen ensures that administrators can maintain a secure, organized, and well-functioning user ecosystem while supporting smooth app operations.

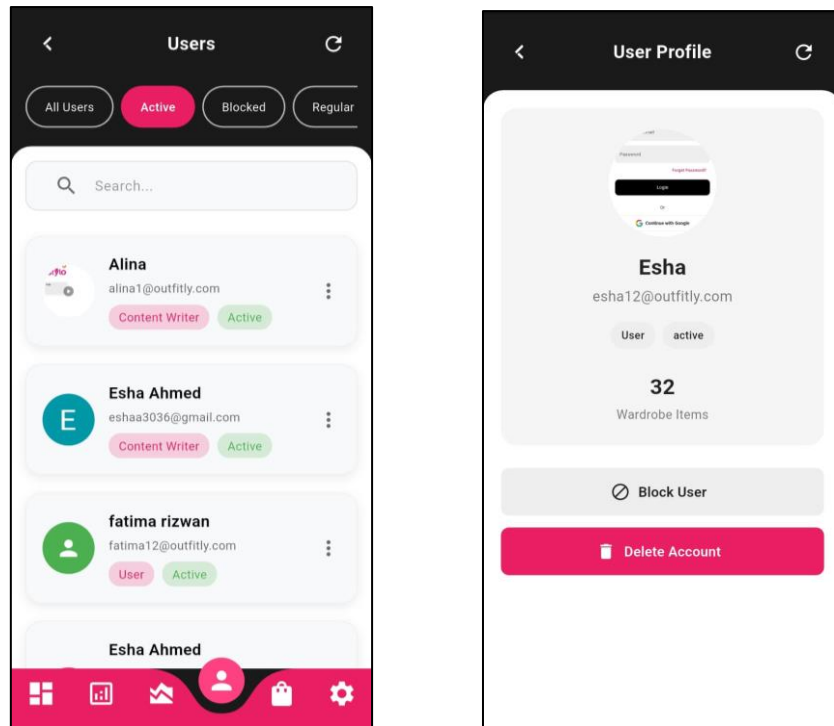


Figure 5.25: User Management Screen

5.3.26. Content Management Screen

Figure 5.26 shows the Content Management Screen, where administrators can review, approve, or reject articles submitted by writers. Approved content is published to the Fashion Feed, while rejected submissions can be returned with feedback for improvement.

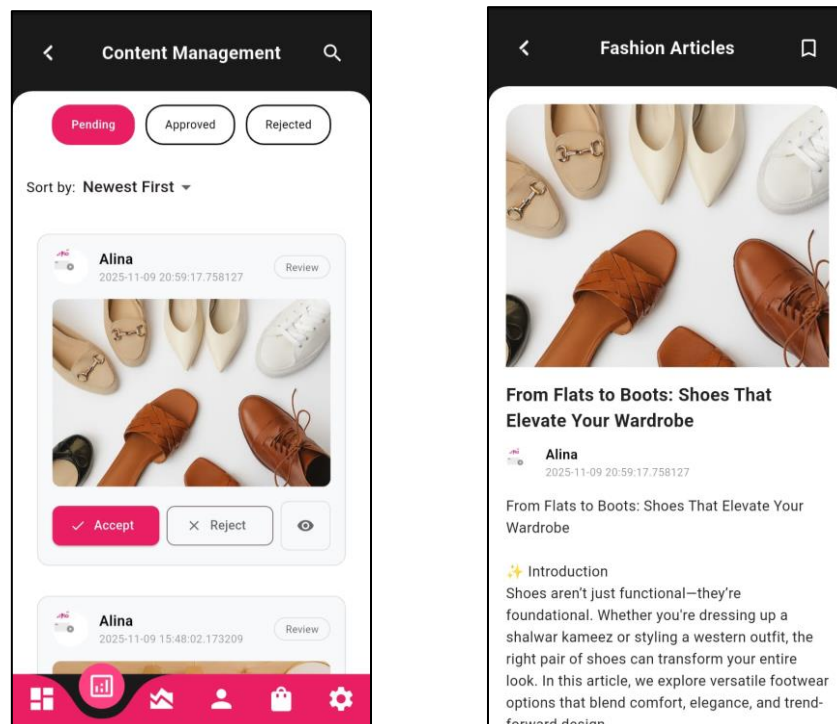


Figure 5.26: Content Management Screen

5.3.27. Manage Shopping Screen

Figure 5.27 The Manage Shopping Screen allows administrators to oversee and maintain the app's shopping module. Admins can add new fashion brands by entering brand details and website links, ensuring users have access to updated and relevant shopping options. They can also delete outdated or inactive brands, maintaining the quality and accuracy of the shopping recommendations. This screen provides an organized interface for managing brand listings efficiently, ensuring a smooth and up-to-date shopping experience for users.

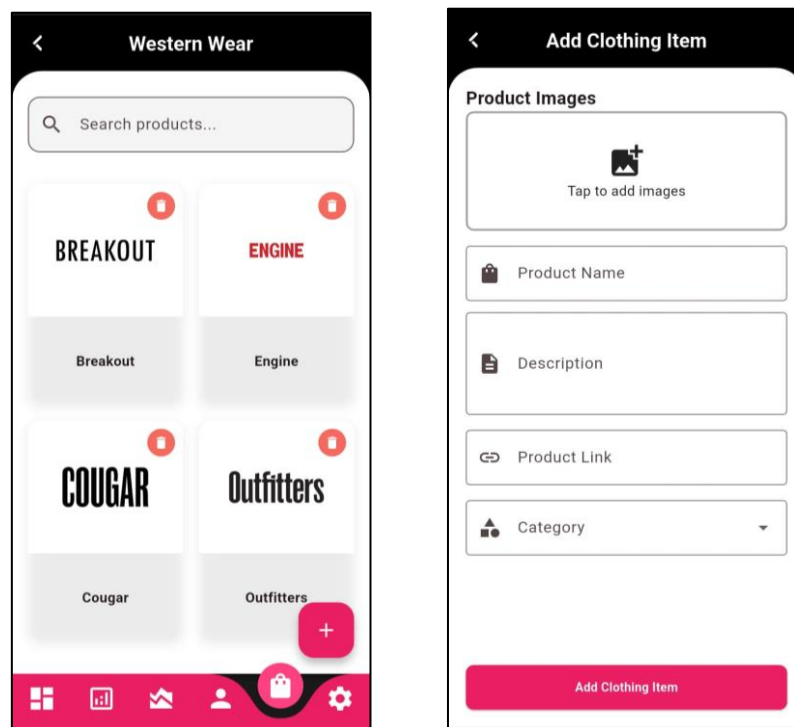


Figure 5.27: Manage Shopping Screen

5.3.28. Analytics Screen

Figure 5.28 illustrates the Analytics Screen, which offers administrators a comprehensive overview of app usage and user engagement. The screen presents key metrics such as daily active users, total registered users, interactions with AI outfit recommendations, and content engagement, providing valuable insights into overall platform performance. Graphs and visualizations display trends over daily, 7-day, 15-day, and monthly periods, enabling administrators to identify usage patterns, monitor growth, and detect anomalies. The screen also supports filtering and segmentation by user demographics, modules, or specific activities, helping admins make informed, data-driven decisions to enhance user experience, optimize features, and guide future app improvements.

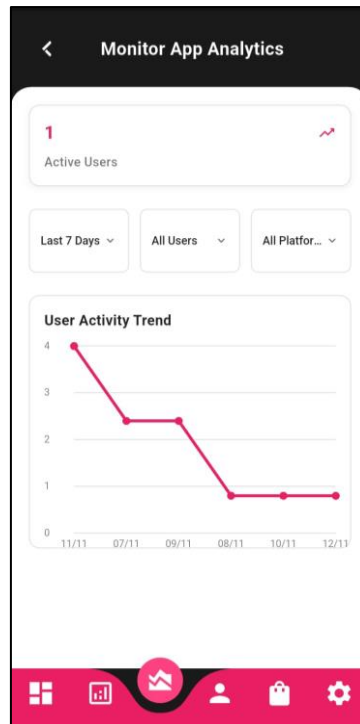


Figure 5.28: Analytics Screen

Chapter 6

TESTING AND EVALUATION

6. Testing and Evaluation

Testing was a crucial phase to ensure the reliability, usability, and performance of the Outfitly app. It involved unit, integration, system, and user acceptance testing (UAT) to verify that all components functioned according to requirements. Key areas tested included wardrobe digitization, AI outfit recommendations, API integrations with Hugging Face and OpenWeather, Firestore operations, security, and overall usability. The process helped identify and resolve defects early, ensuring a seamless, intuitive, and secure experience across all modules, including outfit planning, content delivery, and virtual try-on.

6.1. Manual Testing

Testing was a crucial phase to ensure the reliability, usability, and performance of the Outfitly app. It involved unit, integration, system, and user acceptance testing (UAT) to verify that all components functioned according to requirements. Key areas tested included wardrobe digitization, AI outfit recommendations, API integrations with Hugging Face and OpenWeather, Firestore operations, security, and overall usability. [16] The process helped identify and resolve defects early, ensuring a seamless, intuitive, and secure experience across all modules, including outfit planning, content delivery, and virtual try-on. Performance testing was also conducted to evaluate the app's responsiveness under different loads, while user feedback during UAT helped refine the interface and enhance the overall user experience.

6.1.1. System Testing

Manual testing involves testers interacting with the Outfitly app to simulate real user behaviour and verify functionality. Key features, such as user registration, wardrobe management, AI outfit recommendations, virtual try-on, and smart shopping, are tested to ensure proper operation. This approach helps identify usability issues, interface errors, and module interactions that automated tests may miss, providing valuable insights to enhance the overall user experience.

6.1.2. Unit Testing

Unit testing focuses on individual modules to ensure each feature works correctly in isolation. Key components tested include user registration, AI fashion assistant, virtual try-on, wardrobe digitization, and smart shopping. These tests verify internal logic, such as handling user inputs,

generating outfit suggestions, and validating credentials, allowing issues to be identified and resolved early in development.

Unit Testing 1: Login as a User

Objective: To ensure the user login form is functioning properly and that users can access their dashboard using valid credentials.

Table 6.1: Login as User

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-001	User Login by entering the username and password	Username: mahamahmad575@gmail.com Password: maham123 Username: eshaa3036@gmail.com Password: esha123	User successfully logged in to the system	Pass

Unit Testing 2: User Registration

Testing Objective: To verify that new users can register and select roles securely.

Table 6.2: User Registration

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-002	User Registration	Enter email, password, select role (User/Writer), set PIN or security question	User account created successfully and confirmation message displayed	Pass

Unit Testing 3: Viewing Wardrobe Item Details

Testing Objective: To ensure that detailed information of a wardrobe item loads correctly when a user selects an item.

Table 6.3: View Wardrobe Item Details

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-003	View Wardrobe Item Details	User clicks on "Blue Denim Jacket" item	Item image, colour, fabric, subcategory, wash care, and usage info are displayed	Pass

Unit Testing 4: Generate Outfit Recommendation

Testing Objective: To ensure AI suggests relevant outfits based on outfit colour similarity and user preferences.

Table 6.4: Generate Outfit Recommendation

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-004	Generate Outfit Recommendation	User uploads shirts, pants and shoes images	AI suggests compatible outfits with compatibility score.	Pass

Unit Testing 5: Fashion Feed Interaction

Testing Objective: To verify that users can view, like, comment, and share posts in the fashion feed.

Table 6.5: Fashion Feed Interaction

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-005	Fashion Feed Interaction	User clicks on post, likes, comments	Post displays properly and interactions are recorded	Pass

Unit Testing 6: Schedule Outfit

Testing Objective: To ensure users can schedule and track outfits in the planner.

Table 6.6: Schedule Outfit

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-006	Schedule Outfit	User selects outfit and date in calendar	Outfit is added to planner, reminder set, and tracked in history	Pass

Unit Testing 7: Add New Wardrobe Item

Testing Objective: To ensure that users can successfully log out from the system and end their active session.

Table 6.7: Add New Wardrobe Item

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-007	Add New Wardrobe Item	User uploads item image, enters name, colour, fabric, category, season, subcategory, and wash care	New item is successfully added to wardrobe and displayed in the closet with correct details	Pass

Unit Testing 8: Update User Profile

Testing Objective: To ensure that users can successfully log out from the system and end their active session.

Table 6.8: Update User Profile

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-008	Update User Profile	Edit username, profile picture, style preferences.	Changes are saved and displayed correctly in the profile	Pass

Unit Testing 9: Create/Edit Article

Testing Objective: To ensure writers can create, edit, and publish articles.

Table 6.9: Create/Edit Article

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-009	Create/Edit Article	Writer enters title, content, images, tags	Writer enters title, content, images, tags	Pass

Unit Testing 10: Logout as a User

Testing Objective: To ensure that users can successfully log out from the system and end their active session.

Table 6.10: Logout as User

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-010	Click the logout button and log out	Logout Action	User successfully exited from the system.	Pass

6.1.3. Functional Testing

Functional Testing ensures that all modules within the *Outfitly Mobile Application* operate according to the defined functional requirements and deliver the intended results under specified conditions. Each test case is designed to verify that user-provided inputs and interactions produce the expected outputs, and that data flows correctly between different system modules. As illustrated in Table 6.11, functional test cases were executed for all core features of the application, including authentication, wardrobe management, and outfit recommendations.

Table 6.11: Functional Testing

No.	Test case	Attribute and value	Expected result	Result
1	Log in as a "FYP Committee" member	Username: L001 Password: 1234	The main page for the FYP Committee	Pass

			member is loaded with the FYP Committee navigation bar.	
2	Login as "Admin"	Username: admin001 Password: adminpass	The Admin Dashboard is loaded for managing users, content, and system monitoring.	Pass
3	Login as "Regular User"	Username: user001 Password: password123	The User Dashboard opens, showing wardrobe overview, outfit recommendations, and style feed navigation bar.	Pass
4	Login as "Writer"	Username: writer001 Password: writerpass	The Writer Dashboard appears with controls for viewing submissions, likes, comments, and publishing articles.	Pass

6.1.4. Integration Testing

Integration Testing ensures that all modules and components of the Outfitly Mobile Application interact and function together seamlessly. It focuses on verifying that data flows correctly between features such as user authentication, wardrobe digitization, AI-based outfit recommendations, virtual try-on, fabric care advisor, shopping integration, and content

management. The goal is to ensure that when modules are combined, they operate as a cohesive system without interface or communication issues. As presented in Table 6.12, integration test cases were performed to validate the communication and interaction between these modules.

Table 6.12: Integration Testing

No.	Test case	Attribute and value	Expected result	Result
1	Log in as a "FYP Committee" member	Username: L001 Password: 1234	The main page for the FYP Committee member is loaded with the FYP Committee navigation bar.	Pass
2	Add new clothing item	Action: Upload clothing image and tag details	Clothing item is successfully uploaded, categorized, and displayed in the user's wardrobe.	Pass
3	Generate AI outfit recommendation	Input: User preferences and uploaded wardrobe data	AI successfully suggests personalized outfits based on colour	Pass
4	Use Virtual Try-On feature	Action: Open camera, upload dress image, adjust overlay	The virtual try-on interface opens, overlay image appears on live camera feed, and user can capture preview image.	Pass
5	Access Fabric Care Advisor	Action: Select a clothing item and tap "Care Info"	System displays correct fabric care details and washing instructions for the selected item.	Pass
6	AI Chatbot (Virtual Stylist Assistant)	Action: User interacts with chatbot → Sends query like "Suggest outfit for an	The chatbot successfully communicates with both external AI APIs and Firestore to deliver	Pass

		interview”, Chatbot calls Hugging Face API and fetches wardrobe data from Firestore	personalized outfit suggestions within the chat interface.	
7	Smart Shopping suggestion	Action: Choose “Shop Similar” from wardrobe item	The system suggests related products and redirects the user to partner brand pages.	Pass
8	Read and comment on article	Action: Open Content Hub, Select Article, Add Comment	Comment successfully added and displayed under the selected article in real time.	Pass
10	Write/Edit Article	Action: Publish new article from Writer Dashboard	Published article appears immediately in the Fashion Feed after approval and becomes visible to users.	Pass
11	Admin moderation	Action: Admin manages the users and approves or rejects the content	The user will be blocked successfully and the content will be approved or reject.	Pass

6.1.5. Automated Testing

The Table 6.13 contains environment configuration settings and technology stacks used for the Outfitly Mobile Application, including backend frameworks, databases, APIs, AI integrations,

Table 6.13: Automated Testing

Tool Name	Description	Applied On (FR / Module)	Test Cases	Result
Flutter Integration Test	End-to-end testing tool for validating full	Authentication Module, Wardrobe Digitization Module	Verified complete workflows including user registration, login,	Pass

	app workflows.		password reset, clothing item upload, and outfit generation to ensure seamless module integration.	
Firebase Test Lab	Cloud-based testing on multiple real Android devices.	Entire mobile app UI/UX and Firebase-dependent features	Conducted automated UI stability tests, and device compatibility checks across multiple devices to ensure app stability.	Pass
Mockito / Mocking Libraries	Simulates API and Firestore responses during unit testing	AI Outfit Recommendation, Weather-Based Styling	Mocked API and Firestore responses to test outfit suggestion logic, weather-based clothing recommendations, and error handling.	Pass
Firebase Emulator Suite	Local simulation of Firebase services to test backend	Content Hub, Authentication Module	Verified authentication workflows, Firestore CRUD operations for wardrobe and articles, and content interactions.	Pass
Postman / REST Client	API testing tool for verifying external API calls	AI Outfit Recommendation, Chatbot, Skin Tone Detection	Sent requests to APIs to validate responses and integration with the app.	Pass
Vercel Analytics Testing	Monitors API calls and tracks user events	Analytics & AI personalization	Monitored user interactions, and AI personalization events to ensure accurate analytics tracking.	Pass

Chapter 7
**CONCLUSION AND FUTURE
WORK**

7. Conclusion

The Outfitly Mobile Application represents a comprehensive, AI-driven solution designed to simplify outfit planning, wardrobe management, and personal styling. By integrating multiple intelligent modules such as Wardrobe Digitization, AI-Based Outfit Recommendation, AI Fashion Assistant, Colour Palette Advisor, Smart Shopping, and Virtual Try-On, the system provides users with a holistic and seamless fashion management ecosystem. The application bridges the gap between technology and fashion, leveraging artificial intelligence, image recognition, and personalization to enhance users' daily dressing experiences. It is built using Flutter for cross-platform development, Firebase for backend management, and Hugging Face TinyLlama API for conversational and recommendation intelligence.

Outfitly demonstrates efficiency, scalability, and modern design aesthetics. The platform successfully combines convenience, personalization, and sustainability, making fashion management more digital, intelligent, and user-centered. It not only saves users time and effort but also empowers them to make confident fashion decisions, promotes mindful clothing usage, and encourages smarter wardrobe planning. Overall, Outfitly redefines how individuals interact with their personal style, turning everyday dressing into a smart, enjoyable, and curated experience.

7.1. Future Work

While Outfitly currently delivers an intelligent and seamless wardrobe management experience, several enhancements could further elevate its capabilities, broaden its appeal, and enhance user satisfaction. Potential areas for future development include:

7.1.1. Virtual Try-On with 3D Fit and Body Measurement Integration

Future iterations of Outfitly may incorporate 3D virtual fitting with body scanning or measurement estimation, allowing clothing overlays to accurately conform to each user's body shape and dimensions [14]. AI-driven fabric simulation could visualize texture, drape, and movement, offering a realistic representation of how clothes fit and flow. Augmented Reality (AR) integration would enable users to try outfits in real-time via their smartphone camera or smart mirrors, providing a more immersive and interactive experience [15]. This enhancement would significantly improve convenience, confidence, and satisfaction in online shopping and daily outfit planning, while also reducing the likelihood of purchase returns due to size or fit issues.

7.1.2. Advanced AI Personal Stylist

By integrating deeper AI and machine learning models, Outfitly could evolve into a fully personalized fashion advisor capable of learning individual user preferences over time. The system could generate daily outfit suggestions, create capsule wardrobe collections, and analyse style evolution through predictive analytics. It could proactively provide personalized style alerts, such as reminders for upcoming events, seasonal trends, or wardrobe gaps, ensuring users are always well-dressed and fashion-forward.

7.1.3. Social Styling and Community Engagement

A social networking module could allow users to share outfits, follow fashion creators, and receive feedback from friends or the community. This enhancement would transform Outfitly from a personal assistant into a collaborative fashion platform [7]. Users could participate in styling challenges, vote on trending looks, and showcase their creativity through user-generated outfit boards. Integration of influencer collaborations and live styling sessions could further increase engagement and brand partnerships.

7.1.4. Region- and Culture-Specific Outfit Suggestions

Outfitly could be enhanced to provide personalized outfit recommendations based on the user's region, culture, and local climate. By analyzing regional fashion trends, traditional attire, and seasonal variations, the system could suggest outfits that are both stylish and culturally appropriate. AI could also consider festivals, ceremonies, or cultural events, offering users relevant clothing options for special occasions. This module would make the app more globally adaptable, ensuring recommendations are context-aware and meaningful, while also promoting cultural sensitivity in fashion planning.

7.1.5. Multi-Language and Regional Support

To reach a broader audience, the app could include multi-language support, regional fashion insights, and localized seasonal suggestions. This would make Outfitly accessible to global users with personalized cultural preferences. Incorporating automatic language detection and translation would ensure a seamless user experience across regions. The system could also adapt outfit suggestions based on local climates, cultural attire norms, and regional festivals

Chapter 8

REFERENCE

8. References

- [1] T. M. Choi, "Digitalization in fashion retailing: Data-driven sustainability insights," *IEEE Engineering Management Review*, vol. 49, no. 3, p. 28–36, 2021.
- [2] H. T. Dinh, C. Lee, D. Niyato and P. Wang, "A survey of mobile cloud computing: Architecture, applications, and approaches," *Wireless Communications and Mobile Computing*, vol. 13, no. 18, p. 1587–1611, 2013.
- [3] X. Chen, J. Liang and W. Xu, "Outfit recommendation using vision and context-aware AI models," *IEEE Transactions on Multimedia*, vol. 23, p. 515–528, 2021.
- [4] C. M. Bishop, *Pattern Recognition and Machine Learning*, New York: Springer, 2006.
- [5] P. Kaur and G. Singh, "Sustainable fashion consumption and digital wardrobe management," *Journal of Cleaner Production*, vol. 350, p. 131484, 2022.
- [6] J. Nielsen, *Usability Engineering*, Boston: Morgan Kaufmann, 1994.
- [7] H. B. R. A. Services, "AI in Retail: Personalizing Shopper Experience at Scale," Boston, 2016.
- [8] S. Smith and E. Chang, "Using cloud-based services for scalable mobile application development," *IEEE Cloud Computing*, vol. 7, no. 4, p. 48–55, 2020.
- [9] Cooper, Alan; Reimann, Robert; Cronin, David; Noessel, Christopher, *About Face: The Essentials of Interaction Design*, Indianapolis: Wiley, 2014.
- [10] K. He, X. Zhang, S. Ren and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [11] Google Developers, "Building Secure Mobile Apps Using Firebase Authentication," Mountain View, CA, 2019.
- [12] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, Cambridge, MA: MIT Press, 2016.
- [13] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, p. 1735–1780, 1997.
- [14] Y. Wang, T. Zhang and Y. Guo, "Virtual try-on based on deep learning techniques," in *ACM SIGGRAPH Asia*, 2019.

- [15] W. Zeng and Q. Huang, "A survey on clothing recognition, parsing, and retrieval," *Computer Vision and Image Understanding*, vol. 170, p. 3–23, 2018.
- [16] Y. Li, C. Ma and C. Wang, "Intelligent garment matching based on attributes and color theory," *Expert Systems with Applications*, vol. 157, p. 113456, 2020.
- [17] H. Al-Kilidar, K. Cox and B. Kitchenham, "The use and usefulness of the ISO/IEC 9126 quality standard," *Software Quality Journal*, vol. 13, no. 3, p. 257–270, 2005.
- [18] E. Pantano and M. Viassone, "Engaging consumers on fashion websites: The impact of personalization," *Journal of Fashion Marketing and Management*, vol. 19, no. 2, p. 126–139, 2015.
- [19] Y. Taigman, M. Yang, M. Ranzato and L. Wolf, "DeepFace: Closing the gap to human-level performance in face verification," in *CVPR*, 2014.
- [20] McKinsey Global Institute, "Fashion Forward: AI-Powered Innovation in Apparel," New York, 2020.
- [21] DucNguyen168, "Kaggle," 2023. [Online]. Available: <https://www.kaggle.com/datasets/ducnguyen168/dataset-skin-tone>. [Accessed 5 10 2025].
- [22] A. Ali, "Medium," 23 9 2024. [Online]. Available: <https://medium.com/@atif99478/how-to-choose-the-right-fashion-colors-for-your-skin-tone-e3d8d7482edc>. [Accessed 5 10 2025].
- [23] "GeeksforGeeks," [Online]. Available: <https://www.geeksforgeeks.org/dbms/cosine-similarity/>. [Accessed 5 9 2025].